



Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Πολυτεχνική Σχολή
Τμήμα Ηλεκτρολόγων Μηχανικών &
Μηχανικών Υπολογιστών
Τομέας Ηλεκτρονικής και Υπολογιστών

Διπλωματική Εργασία

Δημιουργία Ιδιωτικού Συστήματος Επιδιόρθωσης Ευπαθειών Ασφαλείας με τη Χρήση Τοπικών Γλωσσικών Μοντέλων.

Εκπόνηση:
Κούνσολας Χρήστος
ΑΕΜ: 10345

Επίβλεψη:
Καθ. Συμεωνίδης Ανδρέας
Υπ. Δρ. Γιώργος Σιαχάμης
Υπ. Δρ. Δημοσθένης Νάτσος

Θεσσαλονίκη, Δεκέμβρης 2025

ΕΥΧΑΡΙΣΤΙΕΣ

Ξεκινώντας θα ήθελα να εκφράσω τις βαθύτερες ευχαριστίες μου σε όλους όσους με στήριξαν σε όλη την διάρκεια της παρούσας διπλωματικής εργασίας.

Αρχικά θα ήθελα να ευχαριστήσω τον κ.Ανδρέα Συμεωνίδη για την εμπιστοσύνη που μου έδειξε, αναθέτοντας μου την συγκεκριμένη εργασία, καθώς και για τις απαραίτητες γνώσεις που παρείχε μέσω των διαλέξεών του.

Επιπλέον θα ήθελα να ευχαριστήσω τους Υποψήφιους Διδάκτορες του τμήματος, Γιώργο Σιαχάμη και Δημοσθένη Νάτσο για την συνεχή και αρίστη επικοινωνία αλλά και την υπομονή τους καθ'όλη την διάρκεια της διπλωματικής εργασίας. Η καθοδήγηση και οι γνώσεις τους αποτέλεσαν κομβικές για την ολοκλήρωση της.

Τελευταίο αλλά εξίσου σημαντικό, ένα μεγάλο ευχαριστώ στην οικογένεια μου και στους φίλους μου, για την στήριξη που μου έδειξαν σε αυτή τη δύσκολη περίοδο και που πάντα πίστευαν σε μένα.

Περίληψη

Στη σημερινή εποχή, τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) όπως το ChatGPT έχουν αλλάξει σε μεγάλο βαθμό τη διαδικασία παραγωγής λογισμικού, καθώς παρέχουν ένα πολύ γρήγορο και αποδοτικό τρόπο συγγραφής κώδικα. Μια από τις ερευνητικές κατευθύνσεις που είναι σε πρώιμο στάδιο είναι η χρήση των LLMs προκειμένου να αντιμετωπιστούν ζητήματα ασφαλείας.

Παρόλα αυτά, το γεγονός ότι τα περισσότερα μοντέλα βρίσκονται διαθέσιμα στο Cloud, εν μέρει λόγω των τεράστιων υπολογιστικών απαιτήσεων τους, αυτόματως εγείρει ερωτήματα εμπιστευτικότητας και ιδιωτικότητας. Για αυτό το λόγο έχει παρατηρηθεί ένα αυξανόμενο ερευνητικό ενδιαφέρον ως προς τα μοντέλα που μπορούν να τρέξουν τοπικά, τα οποία παρέχουν παρόμοιες δυνατότητες με τα μοντέλα που τρέχουν στο Cloud, αλλά χωρίς να απαιτούν την αποστολή ευαίσθητων δεδομένων σε εξωτερικούς διακομιστές.

Ακολουθώντας την παραπάνω τάση, στόχος αυτής της διπλωματικής είναι να δημιουργήσει ένα σύστημα το οποίο θα διενεργεί ελέγχους ασφαλείας (Static Application Security Testing) σε διαδικτυακές εφαρμογές και θα τροφοδοτεί τα αποτελέσματα σε ένα τοπικά εγκατεστημένο LLM, προκειμένου να διορθώσει τις εντοπισμένες ευπάθειες.

Λόγω των περιορισμένων πόρων των τοπικών LLMs, η παραγωγή των κατάλληλων διορθώσεων κώδικα δεν θα πραγματοποιείται σε ένα στάδιο, αλλά θα γίνεται μέσω μιας διαδικασίας πολλαπλών βημάτων. Αρχικά το μοντέλο θα προσπαθεί να κατανοήσει το πρόβλημα, στη συνέχεια θα παράγει τεστ εισοδου-εξοδου με σκοπό να καθοδηγήσουν το μοντέλο στην παραγωγή της τελικής διόρθωσης και τέλος θα διαλέγει την καλύτερη διόρθωση, αφού πρώτα έχει παράξει και αξιολογήσει πολλές πιθανές λύσεις.

Τα αποτελέσματα της διπλωματικής εργασίας δείχνουν ότι η μεθοδολογία που αναπτύχθηκε παρουσιάζει αρκετά υποσχόμενα αποτελέσματα, καθώς σε ευπαθείς κώδικες που εξετάστηκαν, το σύστημα κατάφερε να εντοπίσει τις ευπάθειες τους και σε ένα μεγάλο ποσοστό των περιπτώσεων, να παράξει επιδιορθώσεις που ήταν λειτουργικές και ασφαλείς. Το παραπάνω είναι ένα πρώτο βήμα για την αξιολόγησή της μεθοδολογίας που αναπτύχθηκε, καθώς και για την κατανόηση των δυνατοτήτων των τοπικών LLMs στην αντιμετώπιση ζητημάτων ασφαλείας σε κώδικα.

Title

Creation of a Private Security Vulnerability Repair System Using Local Language Models.

Abstract

In today's era, Large Language Models (LLMs) such as ChatGPT have significantly transformed the software development process by providing a very fast and efficient way to write code. One of the research directions that is still in its early stages is the use of LLMs to address security issues.

However, the fact that most models are available in the Cloud, partly due to their enormous computational requirements, automatically raises questions of confidentiality and privacy. For this reason, there has been growing research interest in models that can run locally, which provide similar capabilities to Cloud-based models but without requiring sensitive data to be sent to external servers.

Following this trend, the aim of this thesis is to create a system that performs security checks (Static Application Security Testing) on web applications and feeds the results into a locally installed LLM in order to fix the identified issues.

Due to the limited resources of local LLMs, the generation of appropriate code fixes will not be performed in a single stage but will be done through a multi-step process. Initially, the model will attempt to understand the problem, then it will generate input-output tests to guide the model in producing the final fix, and finally, it will select the best fix after generating and evaluating multiple possible solutions.

The results of the thesis show that the developed methodology presents quite promising results, as in vulnerable codes that were examined, the system managed to identify their vulnerabilities and in a large percentage of cases, to produce fixes that were functional and secure. The above is a first step for the evaluation of the developed methodology, as well as for understanding the capabilities of local LLMs in addressing security issues in code.

Kounsolas Christos

Electrical & Computer Engineering Department,
Aristotle University of Thessaloniki, Greece

December 2025

Περιεχόμενα

Ευχαριστίες	ii
Περίληψη	iv
Abstract	vi
Ακρωνύμια	xiii
1 Εισαγωγή	1
1.1 Περιγραφή του Προβλήματος	2
1.2 Σκοπός - Συνεισφορά της Διπλωματικής Εργασίας	2
1.3 Διάρθρωση της Αναφοράς	3
2 Επισκόπηση της Ερευνητικής Περιοχής και Τεχνολογικό Υπόβαθρο	4
2.1 Προβλήματα και Προκλήσεις	6
2.2 Prompt Engineering	8
2.3 LLM-based Automatic Program Repair (APR)	11
2.4 Patch Selection and Validation	17
3 Μεθοδολογία και Υλοποίηση	19
3.1 Βασικό Μοντέλο και SAST εργαλείο	20
3.1.1 Μοντέλο: NextCoder-32B-AWQ	20
3.1.2 Εργαλείο Στατικής Ανάλυσης: Bearer	24
3.2 Βασικοί Άξονες Σχεδίασης	24
3.3 Η Αρχιτεκτονική της Εφαρμογής	26
3.3.1 Περιγραφή Σταδίων Αρχιτεκτονικής	26
3.3.2 Αναλυτική Περιγραφή της Ροής Εργασίας	26
3.4 Παραμετροποίηση του τοπικού LM	29
3.4.1 Παράμετροι χρήσης του LM	29
3.5 Πείραμα 1: AI-Generated Ευπαθείς Ιστοσελίδες	30
3.6 Πείραμα 2: YesWeHack Ευπαθής Κώδικας	31
4 Πειραματικά Αποτελέσματα	33
4.1 Dataset	34
4.1.1 AI-Generated Dataset	34
4.1.2 YesWeHack Dataset	34
4.2 Αποτελέσματα	36
4.2.1 Αποτελέσματα στα Απλά Παραδείγματα Ευπαθούς Κώδικα	36
4.2.2 Αποτελέσματα στα Παραδείγματα Ευπαθούς Κώδικα από YesWeHack	36
4.3 Παράδειγμα Διόρθωσης	38
4.3.1 Αρχικός Ευπαθής Κώδικας	39

4.3.2	Κατανόηση της Ευπάθειας (Self Reflection)	40
4.3.3	Παραγωγή Δοκιμών Εισόδου-Εξόδου	41
4.3.4	Παραγωγή Επιδιορθώσεων (Patch Generation)	41
4.3.5	Επιλογή της Τελικής Επιδιόρθωσης	43
5	Συμπεράσματα	44
5.1	Γενικά Συμπεράσματα	44
5.2	Περιορισμοί	47
6	Μελλοντικές Επεκτάσεις	48
	Βιβλιογραφία	50

Κατάλογος Σχημάτων

2.1	Επίδραση της Focal Loss και του Sample Weighting στην απόδοση του μοντέλου.	6
2.2	Chain of Thought Prompting	9
2.3	Η μέθοδος Zero Shot Chain of Thought	9
2.4	Η μέθοδος Tree Of Thought	10
2.5	Βελτίωση της απόδοσης εντοπισμού ευπαθειών με χρήση LSAST	10
2.6	Φάση Συλλογής του ThinkRepair	12
2.7	Φάση Επιδιόρθωσης του ThinkRepair	13
2.8	Αποτελέσματα ThinkRepair σε σύγκριση με άλλες προσεγγίσεις	14
2.9	Ροή εργασίας του SkipAnalyzer	15
2.10	Ροή εργασίας του AlphaCodium	16
2.11	Αποτελέσματα χρήσης εντροπίας για την επιλογή patches	17
3.1	Ο αλγόριθμος SeleKT	21
3.2	Σύγκριση απόδοσης μοντέλων σε εργασίες προγραμματισμού.	23
3.3	Το logo του Bearer.	24
3.4	Η Αρχιτεκτονική της Εργασίας	26
3.5	Η Αρχιτεκτονική της Εργασίας	27
4.1	Κατανομή των ευπαθειών που επιδιορθώθηκαν ανά τύπο ευπάθειας	38
5.1	Επίδραση των σταδίων στην απόδοση του εργαλείου	45

Κατάλογος Αποσπασμάτων

4.1	Ευπαθής κώδικας SQL Injection	39
4.2	Παραγόμενα test cases για την SQLi ευπάθεια	41

Κατάλογος πινάκων

4.1	Αποτελέσματα στα απλά παραδείγματα ευπαθούς κώδικα	36
4.2	Αποτελέσματα στα παραδείγματα ευπαθούς κώδικα από YesWeHack	37
5.1	Σύγκριση αρχιτεκτονικής και απλού μοντέλου στο Πείραμα2	46
5.2	Εντροπίες των σωστών επιδιορθώσεων για κάθε αρχείο που επιδιορθώθηκε	47

Ακρωνύμια Εγγράφου

Παρακάτω παρατίθενται ορισμένα από τα πιο συχνά χρησιμοποιούμενα ακρωνύμια της παρούσας διπλωματικής εργασίας:

AI	→	Artificial Intelligence
APR	→	Automatic Program Repair
AWQ	→	Activation-aware Weight Quantization
CLI	→	Command Line Interface
CoT	→	Chain-of-Thought
CWE	→	Common Weakness Enumeration
DAST	→	Dynamic Application Security Testing
LLM	→	Large Language Model
NLP	→	Natural Language Processing
NMT	→	Neural Machine Translation
SAST	→	Static Application Security Testing

1

Εισαγωγή

Η Τεχνητή Νοημοσύνη (Artificial Intelligence - AI) έχει εξελιχθεί ραγδαία τα τελευταία χρόνια, με εφαρμογές που εκτείνονται σε πολλούς τομείς της τεχνολογίας και της επιστήμης. Στο επίκεντρο αυτής της εξέλιξης βρίσκονται τα Μεγάλα Γλωσσικά Μοντέλα (Large Language Models, LLMs), τα οποία έχουν αποδείξει την ικανότητά τους να κατανοούν και να παράγουν ανθρώπινη γλώσσα και κώδικα [1] με εντυπωσιακή ακρίβεια, καθιστώντας τα πολύτιμα εργαλεία για πληθώρα εφαρμογών. Μια εφαρμογή των συγκεκριμένων μοντέλων είναι στη διαδικασία παραγωγής λογισμικού, καθώς παρέχουν έναν πολύ γρήγορο και αποδοτικό τρόπο συγγραφής κώδικα. Ταυτόχρονα όμως, η χρήση των LLMs και η αυτοματοποίηση της παραγωγής κώδικα φέρνει στο προσκήνιο νέες προκλήσεις και ανησυχίες σχετικά με την ασφάλεια του παραγόμενου λογισμικού. Έτσι η ανάγκη για αυτοματοποιημένες λύσεις εντοπισμού και διόρθωσης ευπαθειών είναι πιο επιτακτική από ποτέ. Λόγω της παραπάνω κατάστασης, μία από τις πιο ενδιαφέρουσες και επίκαιρες εφαρμογές των LLMs είναι η χρήση αυτών των μοντέλων προκειμένου να αντιμετωπιστούν ευπάθειες και ζητήματα ασφαλείας στον κώδικα. Ως επί το πλείστον, η σάρωση για ευπάθειες σε λογισμικό γινόταν με SAST (Static Application Security Testing) και DAST (Dynamic Application Security Testing) εργαλεία. Τα SAST εργαλεία αναλύουν τον πηγαίο κώδικα για να εντοπίσουν πιθανά ζητήματα ασφαλείας χωρίς να τον εκτελούν ενώ τα DAST εργαλεία δοκιμάζουν την εφαρμογή κατά τη διάρκεια της εκτέλεσής της για να εντοπίσουν ευπάθειες. Με την δημιουργία και βελτίωση των LLMs όμως, εμφανίστηκε και η δυνατότητα χρησιμοποίησής τους στην ανίχνευση και διόρθωση ευπαθειών σε λογισμικό [2].

1.1 ΠΕΡΙΓΡΑΦΗ ΤΟΥ ΠΡΟΒΛΗΜΑΤΟΣ

Η χρήση όμως των μοντέλων αυτών για την ανίχνευση και διόρθωση ευπαθειών σε λογισμικό δεν είναι χωρίς προκλήσεις και ανησυχίες. Πιο συγκεκριμένα, τα μεγάλα γλωσσικά μοντέλα, λόγω του τεράστιου μεγέθους τους και των απαιτήσεων σε υπολογιστική ισχύ, συνήθως φιλοξενούνται σε υποδομές Cloud. Αυτό αυτομάτως εγείρει ανησυχίες σχετικά με την εμπιστευτικότητα και την ιδιωτικότητα των δεδομένων, ειδικά όταν πρόκειται για ευαίσθητο ή ιδιωτικό κώδικα. Οι παραπάνω ανησυχίες έχουν στρέψει το ενδιαφέρον στην ανάπτυξη και χρήση μικρότερων γλωσσικών μοντέλων που μπορούν να τρέξουν τοπικά, χωρίς να απαιτείται η αποστολή δεδομένων σε εξωτερικούς διακομιστές. Ακόμα και αν η διαφορά στο μέγεθος των τοπικών μοντέλων σε σχέση με τα Cloud-based είναι τεράστια, οι δυνατότητες που προσφέρουν είναι αρκετά ικανοποιητικές για πολλές εφαρμογές, συμπεριλαμβανομένης της ανίχνευσης και διόρθωσης ευπαθειών σε λογισμικό. Επιπλέον, εξαιτίας του σημαντικά μικρότερου μεγέθους τους, η διαδικασία ανίχνευσης και διόρθωσης ευπαθειών ίσως χρειαστεί παραπάνω βήματα και πληροφορίες για να επιτευχθεί το επιθυμητό αποτέλεσμα. Αυτό σημαίνει ότι η σωστή διαμόρφωση των προτροπών (Prompt Engineering) και η χρήση τεχνικών όπως το Chain-of-Thought (CoT) αποτελούν κρίσιμους παράγοντες για την επιτυχία της διαδικασίας.

1.2 ΣΚΟΠΟΣ - ΣΥΝΕΙΣΦΟΡΑ ΤΗΣ ΔΙΠΛΩΜΑΤΙΚΗΣ ΕΡΓΑΣΙΑΣ

Ο κύριος σκοπός της παρούσας διπλωματικής εργασίας είναι η δημιουργία ενός συστήματος, το οποίο θα διενεργεί ελέγχους ασφαλείας σε μια διαδικτυακή εφαρμογή, και θα τροφοδοτεί τα αποτελέσματα σε ένα τοπικά εγκατεστημένο LLM, προκειμένου να διορθώσει τα ζητήματα που έχουν προκύψει από τους παραπάνω ελέγχους. Λόγω του ότι το μέγεθος του μοντέλου θα είναι περιορισμένο, έτσι ώστε να μπορεί να τρέξει τοπικά, θα χρειαστεί η χρήση διαφόρων τεχνικών για την βελτιστοποίηση της διαδικασίας ανίχνευσης και διόρθωσης ευπαθειών. Συγκεκριμένα, θα χρησιμοποιηθούν τεχνικές όπως το Prompt Engineering, το Chain-of-Thought (CoT), το self-reflection και άλλες, ώστε να επιτευχθεί το καλύτερο δυνατό αποτέλεσμα. Τα ερευνητικά ερωτήματα (ΕΕ) που καθοδήγησαν την παρούσα διπλωματική εργασία είναι τα εξής αριθμημένα:

1. **ΕΕ1:** Πόσο αποδοτικό είναι ένα τοπικά εγκατεστημένο SLM για την ανίχνευση και επιδιόρθωση ευπαθειών λογισμικού;
2. **ΕΕ2:** Μπορεί ένα pipeline πολλαπλών βημάτων να βελτιώσει την απόδοση ενός μοντέλου στην επιδιόρθωση ευπαθειών ασφαλείας;
3. **ΕΕ3:** Βοηθάει η εντροπία στο να λαμβάνουμε πιο άμεσα σωστά αποτελέσματα επιδιόρθωσης ευπαθούς κώδικα;
4. **ΕΕ4:** Πως μεταβάλλεται η απόδοση του μοντέλου ανάλογα με τον τύπο της ευπάθειας που προσπαθεί να διορθώσει;

1.3 ΔΙΑΡΘΡΩΣΗ ΤΗΣ ΑΝΑΦΟΡΑΣ

Η διάρθρωση της παρούσας διπλωματικής εργασίας είναι η εξής:

- **Κεφάλαιο 2:** Σε αυτό το κεφάλαιο εξετάζεται η υπάρχουσα βιβλιογραφία, όπου παρουσιάζονται τα βασικά προβλήματα των γλωσσικών μοντέλων και οι τεχνικές που έχουν προταθεί για την αντιμετώπισή τους. Επιπλέον, παρουσιάζονται τεχνικές Prompt Engineering που έχουν χρησιμοποιηθεί για την βελτίωση της απόδοσης των μοντέλων, καθώς και μετρικές που έχουν προταθεί για την αξιολόγηση των διορθώσεων κώδικα που προτείνονται από τα μοντέλα. Τέλος γίνεται αναφορά σε αυτοματοποιημένα συστήματα ανίχνευσης και επιδιόρθωσης κώδικα.
- **Κεφάλαιο 3:** Σε αυτό το κεφάλαιο παρουσιάζεται η αρχιτεκτονική του συστήματος που αναπτύχθηκε για την ανίχνευση και επιδιόρθωση ευπαθειών κώδικα και αναλύονται οι τεχνικές που χρησιμοποιήθηκαν για την βελτιστοποίηση της διαδικασίας.
- **Κεφάλαιο 4:** Σε αυτό το κεφάλαιο παρουσιάζονται τα πειράματα που πραγματοποιήθηκαν για την αξιολόγηση της απόδοσης του συστήματος, καθώς και τα αποτελέσματα που προέκυψαν από αυτά.
- **Κεφάλαιο 5:** Σε αυτό το κεφάλαιο συνοψίζονται τα ευρήματα της διπλωματικής εργασίας και αναφέρονται τα συμπεράσματα που προέκυψαν από την ανάλυση των αποτελεσμάτων, καθώς και οι περιορισμοί της παρούσας εργασίας.
- **Κεφάλαιο 6:** Σε αυτό το κεφάλαιο παρουσιάζονται οι προοπτικές μελλοντικής έρευνας που προκύπτουν από την παρούσα διπλωματική εργασία, καθώς και οι πιθανές βελτιώσεις που μπορούν να γίνουν στο σύστημα που αναπτύχθηκε.

Επισκόπηση της Ερευνητικής Περιοχής και Τεχνολογικό Υπόβαθρο

Η τεχνητή νοημοσύνη έχει γνωρίσει σημαντική ανάπτυξη τα τελευταία χρόνια, με εφαρμογές που εκτείνονται σε ένα ευρύ φάσμα τομέων. Ένα από τα πλέον ενεργά πεδία εφαρμογής αφορά την κατανόηση και παραγωγή κώδικα, όπου μοντέλα μηχανικής μάθησης είναι σε θέση να αναλύουν, να ερμηνεύουν και να παράγουν κώδικα. Στο πλαίσιο αυτό, τα Μεγάλα Γλωσσικά Μοντέλα (Large Language Models – LLMs), όπως το ChatGPT, έχουν επηρεάσει σημαντικά τη διαδικασία ανάπτυξης λογισμικού, παρέχοντας αυτοματοποιημένες και αποδοτικές μεθόδους συγγραφής κώδικα.

Παράλληλα, η συνεχής αύξηση των επιθέσεων σε λογισμικά συστήματα καθιστά επιτακτική την ανάγκη για αποτελεσματικές και αυτοματοποιημένες λύσεις εντοπισμού και διόρθωσης ευπαθειών. Πιο συγκεκριμένα, το παγκόσμιο κόστος από κυβερνοεπιθέσεις εκτιμάται ότι θα κυμανθεί από 8.15 έως 13.82 τρισεκατομμύρια δολάρια μέχρι το 2028, ενώ καταγράφονται κατά μέσο όρο περίπου 26.000 επιθέσεις καθημερινά [3].

Παραδοσιακά, ο εντοπισμός ευπαθειών πραγματοποιείται μέσω εργαλείων στατικής ανάλυσης (Static Application Security Testing – SAST) και δυναμικής ανάλυσης (Dynamic Application Security Testing – DAST), τα οποία βασίζονται είτε στην ανάλυση του πηγαίου κώδικα χωρίς εκτέλεση είτε στην παρακολούθηση της συμπεριφοράς του κατά την εκτέλεση.

Με την εξέλιξη των LLMs, έχει αναδειχθεί η δυνατότητα αξιοποίησής τους στον εντοπισμό και την επιδιόρθωση ευπαθειών λογισμικού. Ωστόσο, το γεγονός ότι τα περισσότερα μοντέλα βρίσκονται διαθέσιμα στο Cloud, εν μέρει λόγω των τεράστιων υπολογιστικών απαιτήσεων, αυτομάτως εγείρει ερωτήματα εμπιστευτικότητας και ιδιωτικότητας, στα οποία θα αναφερθούμε στα επόμενα υποκεφάλαια. Έτσι, παρατηρείται μια στροφή προς τη χρήση τοπικών μικρότερων μοντέλων αλλά και τεχνικών που βελτιώνουν την απόδοσή τους.

Σκοπός αυτού του κεφαλαίου είναι να παρουσιάσει μια επισκόπηση της ερευνητικής περιοχής, εστιάζοντας στα προβλήματα και τις ανησυχίες που προκύπτουν από τη χρήση LLMs για τον εντοπισμό και την επιδιόρθωση ευπαθειών, καθώς και τις σύγχρονες τεχνικές που έχουν αναπτυχθεί για την αντιμετώπιση αυτών των προβλημάτων.

2.1 ΠΡΟΒΛΗΜΑΤΑ ΚΑΙ ΠΡΟΚΛΗΣΕΙΣ

Η χρησιμοποίηση των γλωσσικών μοντέλων όμως δεν έρχεται χωρίς προκλήσεις. Ένα βασικό πρόβλημα κατά τον εντοπισμό ευπαθειών αφορά την αδυναμία κάλυψης πρόσφατα ανακαλυφθεισών ευπαθειών, καθώς τα δεδομένα εκπαίδευσης των μοντέλων περιορίζονται σε μια συγκεκριμένη χρονική περίοδο. Ως αποτέλεσμα, τα μοντέλα στερούνται γνώσης για νεότερα πρότυπα ευπαθειών και σύγχρονες τεχνικές επίθεσης. Παρ' όλα αυτά, με κατάλληλο prompt engineering το LLM μπορεί να καθοδηγηθεί ώστε να παράγει πιο δομημένες και ελέγξιμες προτάσεις επιδιόρθωσης, αξιοποιώντας γενίκευση από γνωστά μοτίβα και καθιερωμένες πρακτικές.

Ένα ακόμα βασικό πρόβλημα είναι η ανισορροπία στην κατανομή των δεδομένων, καθώς στον πραγματικό κόσμο οι ευπαθείς συναρτήσεις αποτελούν μειοψηφία σε σχέση με τις μη ευπαθείς. Η ανισορροπία αυτή καθιστά το πρόβλημα πιο δύσκολο, καθώς οι κλασικές συναρτήσεις κόστους, όπως η cross-entropy loss, οδηγούν το μοντέλο να επικεντρώνεται στην κυρίαρχη κατηγορία. Για την αντιμετώπιση αυτού του ζητήματος, έχουν προταθεί τεχνικές όπως η focal loss [4] και η δυναμική στάθμιση δειγμάτων (sample weighting), ώστε να δίνεται η απαραίτητη έμφαση στις θετικές (ευπαθείς) περιπτώσεις κατά τη φάση της εκπαίδευσης. Ωστόσο, όπως παρουσιάζεται και στην εικόνα 2.1 βλέπουμε ότι η βελτίωση είναι ελάχιστη. Η παράμετρος γ ρυθμίζει τον βαθμό έμφασης στις δύσκολα ταξινομήσιμες περιπτώσεις, οι οποίες προκύπτουν κυρίως λόγω της ανισορροπίας των δεδομένων.

Weight	γ	Val. AUC	Test AUC	Test F1
3	1.0	0.88	0.877	0.273
10	3	0.872	0.853	0.226
10	1	0.877	0.863	0.243
3	3	0.874	0.865	0.286
None	1	0.87	0.873	0.265
None	3	0.87	0.868	0.272

Σχήμα 2.1: Επίδραση της Focal Loss και του Sample Weighting στην αποδοχή του μοντέλου.

Τέλος, ένα ακόμη ουσιώδες εμπόδιο στη χρήση των LLMs αποτελεί το υψηλό υπολογιστικό κόστος που απαιτείται για τη διαδικασία προσαρμογής (fine-tuning). Για τον λόγο αυτό έχει προταθεί η μέθοδος LoRA (Low-Rank Adaptation) [5]. Η τεχνική αυτή επιτρέπει την προσαρμογή μεγάλων μοντέλων μέσω της παραγοντοποίησης των πινάκων βαρών σε προσεγγίσεις χαμηλής τάξης, χωρίς την ανάγκη εκπαίδευσης όλων των παραμέτρων. Ως αποτέλεσμα, μειώνονται σημαντικά τόσο ο αριθμός των εκπαιδευσιμων παραμέτρων όσο και οι απαιτήσεις σε μνήμη, καθιστώντας τη μέθοδο ιδιαίτερα κατάλληλη για περιβάλλοντα με περιορισμένους υπολογιστικούς πόρους.

2.2 PROMPT ENGINEERING

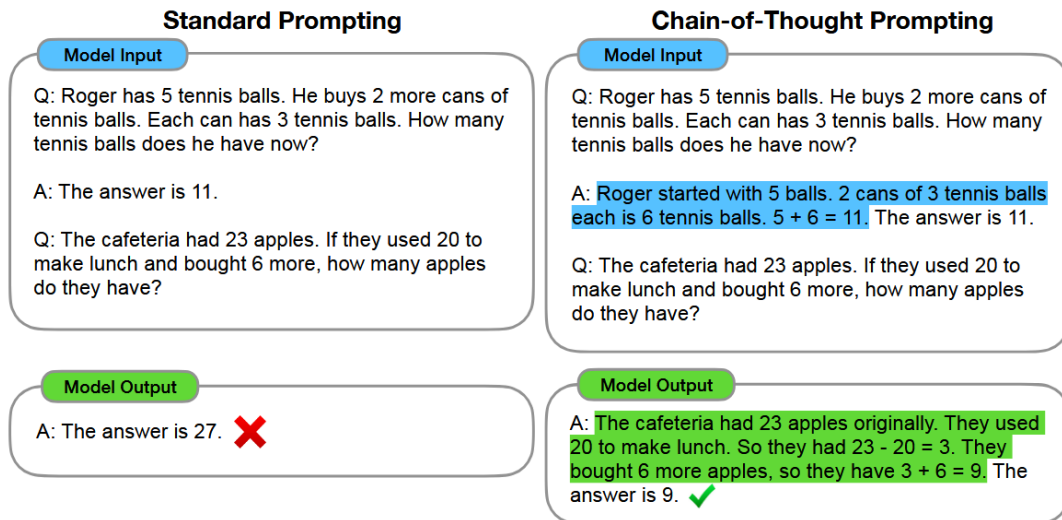
Παρόλο που οι τεχνικές όπως η RAG, η focal loss και η LoRA συμβάλλουν στη βελτίωση της ακρίβειας, της αποδοτικότητας και της δυνατότητας εκπαίδευσης των LLMs, υπάρχει ένα ακόμη σύνολο προσεγγίσεων που επικεντρώνεται όχι στο ίδιο το μοντέλο, αλλά στον τρόπο με τον οποίο το χρησιμοποιούμε. Οι τεχνικές αυτές ανήκουν στο πεδίο του prompt engineering και έχουν στόχο να ενισχύσουν την απόδοση των LLMs μέσω κατάλληλης διαμόρφωσης της εισόδου, χωρίς να απαιτείται τροποποίηση της υποκείμενης αρχιτεκτονικής ή επαναεκπαίδευση του μοντέλου.

Σύμφωνα με πρόσφατες μελέτες [6], η βελτίωση της απόδοσης ενός LLM μέσω της παροχής παραδειγμάτων (n-shot learning), δεν υποδηλώνει ότι το μοντέλο μαθαίνει νέες πληροφορίες ‘άλλα ότι το prompt λειτουργεί ως ένας μηχανισμός ανάκλησης, βοηθώντας το μοντέλο να εντοπίσει σχετικές γνώσεις που έχουν ήδη ενσωματωθεί κατά την φάση της εκπαίδευσης του. Επιπλέον, έχει παρατηρηθεί πως σε ορισμένες περιπτώσεις τα παραδείγματα οδηγούν το μοντέλο να επηρεάζεται υπερβολικά από το περιεχόμενο του παραδείγματος, ερμηνεύοντας το ως μέρος μιας ενιαίας αφηγηματικής ροής, γεγονός που μπορεί να επιδεινώσει την απόδοση (π.χ. χειρότερο αποτέλεσμα στο 1-shot σε σχέση με το 0-shot). Ωστόσο, αυτή η ιδιότητα αξιοποιείται πλέον στρατηγικά μέσω της τεχνικής του in-context learning, όπου η προσεκτική επιλογή παραδειγμάτων που ευθυγραμμίζονται με το ζητούμενο task οδηγεί σε βελτίωση της απόδοσης.

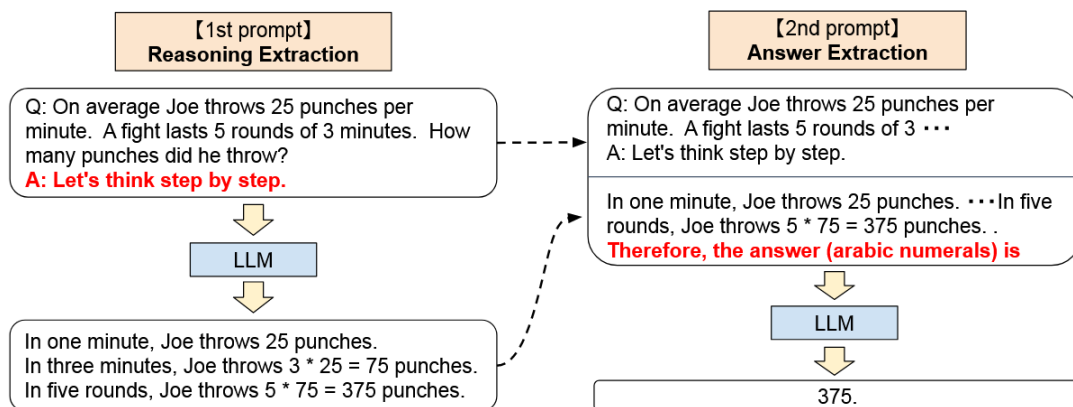
Μια ακόμα τεχνική που ενδέχεται να βελτιώσει την απόδοση του LLM είναι το ChainOfThought prompting [7], το οποίο ενθαρρύνει το μοντέλο να παράγει ενδιάμεσα βήματα σκέψης πριν καταλήξει στην τελική απάντηση. Στην εικόνα 2.2 παρουσιάζεται η τροποποίηση της εισόδου του μοντέλου με βάση την συγκεκριμένη μέθοδο και την σωστή ανταπόκριση του. Παρότι η μέθοδος αυτή έχει αποδειχθεί ιδιαίτερα αποτελεσματική για μεγάλα γλωσσικά μοντέλα, έχει παρατηρηθεί ότι σε μικρότερα μοντέλα μπορεί να οδηγήσει ακόμη και σε υποβάθμιση της απόδοσης σε σύγκριση με το απλό prompting. Αντίθετα, σημαντική βελτίωση παρατηρείται σε LLMs με περισσότερες από 100 δισεκατομμύρια παραμέτρους.

Παρόμοιά προσέγγιση αποτελεί το Zero Shot Chain of Thought prompting [8], όπου αντί να παρέχονται παραδείγματα, το prompt τροποποιείται ώστε να ενθαρρύνει το μοντέλο να παραγάγει μια ακολουθία συλλογισμών πριν απαντήσει στο ερώτημα. Η διαδικασία αυτή επιτρέπει στο μοντέλο να αναπτύξει μια εσωτερική λογική πορεία, η οποία μπορεί να οδηγήσει σε πιο ακριβείς απαντήσεις, ειδικά σε πολύπλοκα προβλήματα. Όπως φαίνεται στην εικόνα 2.3 βλέπουμε ότι η απάντηση εξάγεται σε δύο βήματα, πρώτα παράγεται η αλυσίδα συλλογισμών και μετά η τελική απάντηση.

Ακόμα και με την εφαρμογή των παραπάνω μεθόδων, τα γλωσσικά μοντέλα υστερούν σε εργασίες που απαιτούν διερεύνηση πολλαπλών εναλλακτικών λύσεων, στρατηγικό σχεδιασμό ή όπου οι αρχικές αποφάσεις διαδραματίζουν καθοριστικό ρόλο. Για την αντιμετώπιση αυτών των περιορισμών έχει προταθεί η μέθοδος Tree Of Thought (ToT) [9], η οποία γενικεύει το Chain Of Thought επιτρέποντας την εξερεύνηση πολλαπλών ενδιάμεσων συλλογιστικών διαδρομών για την λύση του προβλήματος, όπως φαίνεται και στην εικόνα 2.4. Κάθε κόμβος στο δέντρο αντιπροσωπεύει μια κατάσταση σκέψης, ενώ κάθε κλάδος αντιπροσωπεύει μια πιθανή



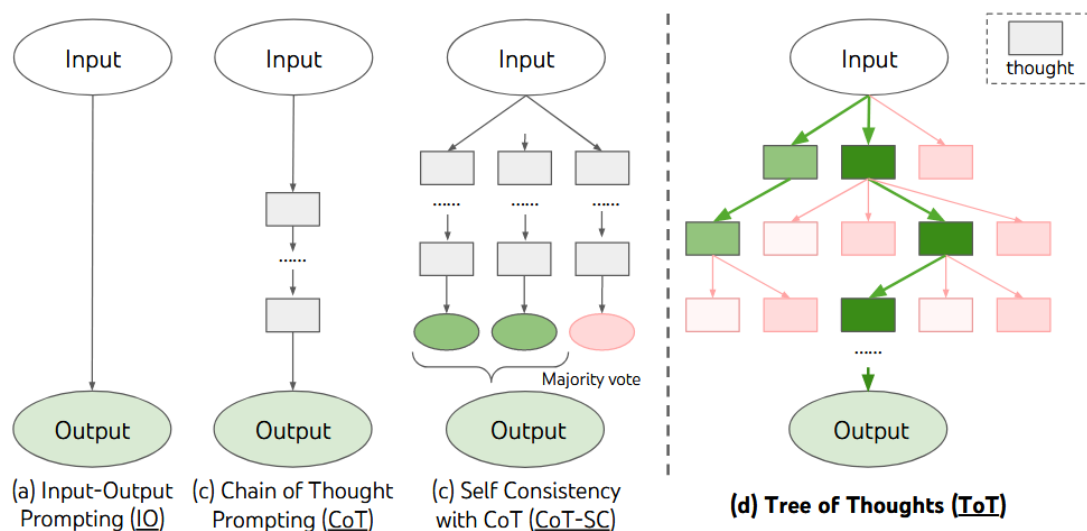
Σχήμα 2.2: Chain of Thought Prompting



Σχήμα 2.3: Η μέθοδος Zero Shot Chain of Thought

ενέργεια ή απόφαση που μπορεί να ληφθεί από το μοντέλο. Με την υλοποίηση αυτής της προσέγγισης, το μοντέλο μπορεί να εξερευνήσει πολλαπλές διαδρομές σκέψης, να αξιολογήσει τις διαφορετικές επιλογές και να επιλέξει την πιο κατάλληλη πορεία δράσης για την επίλυση του προβλήματος.

Σε πιο πρακτικές εφαρμογές όπως ο εντοπισμός ευπαθειών, η χρήση LLMs ενισχύεται μέσα από κατάλληλα σχεδιασμένα prompts. Συγκεκριμένα, έχειδειχθεί ότι όταν τα αποτελέσματα στατικών εργαλείων ανάλυσης (όπως το Bearer) εισάγονται στο prompt ενός μοντέλου (Llama3-70B) μαζί με αριθμημένο πηγαίο κώδικα, επιτυγχάνεται σημαντικά βελτιωμένη ακρίβεια εντοπισμού ευπαθειών (LSAST) σε σύγκριση τόσο με την απλή χρήση SAST εργαλείων όσο και με προσεγγίσεις που βασίζονται στην τεχνική RAG [10]. Όπως φαίνεται στην εικόνα 2.5 ο συνδυασμός SAST με LLM (LSAST) χωρίς RAG επιτυγχάνει τα καλύτερα αποτελέσματα, έχοντας σημαντικά καλύτερη απόδοση από τα SAST εργαλεία (raw), καθώς και από



Σχήμα 2.4: Η μέθοδος Tree Of Thought

την Combined-LSAST προσέγγιση, η οποία εφαρμόζει RAG με similarity searches χρησιμοποιώντας και τις 2 μεθόδους που αναφέρθηκαν πιο πάνω (Functionality, Abstraction) αποπροσανατολίζοντας όμως το μοντέλο. Το εύρημα αυτό υπογραμμίζει τη σημασία της σωστής καθοδήγησης του μοντέλου μέσω του prompt, η οποία μπορεί να αναδείξει τις δυνατότητές του ακόμη και χωρίς την προσθήκη επιπλέον μηχανισμών ανάκτησης γνώσης.

Measure	raw	raw-LSAST	Combined-LSAST
TP-Rate	17.91%	68.89%	35.71%
FP-Rate	100%	100%	100%
Accuracy	24%	62%	32.61%
Precision	63.16%	86.11%	78.95%
F1-Score	38.71%	76.54%	49.18%

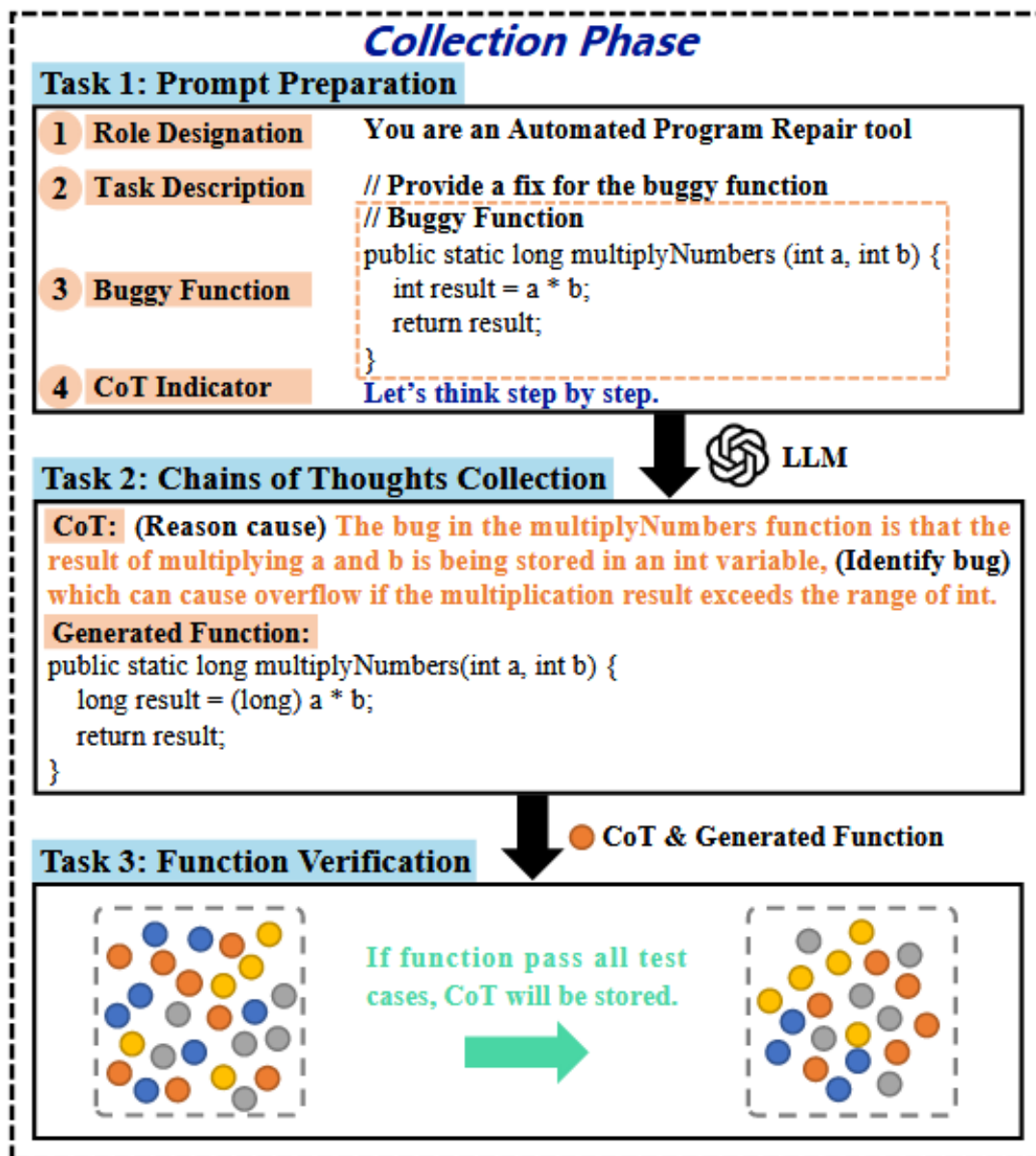
Σχήμα 2.5: Βελτίωση της απόδοσης εντοπισμού ευπαθειών με χρήση LSAST

2.3 LLM-BASED AUTOMATIC PROGRAM REPAIR (APR)

Επομένως βλέπουμε ότι η χρήση των LLMs για την αυτόματη επιδιόρθωση κώδικα αποτελεί μια αρκετά υποσχόμενη κατεύθυνση, όπου οι τεχνικές Prompt Engineering που αναφέρθηκαν παραπάνω βελτιώνουν την ποιότητα των απαντήσεων μέσω κατάλληλης καθοδήγησης του μοντέλου. Ωστόσο, σε εργασίες που παρουσιάζουν μεγαλύτερη πολυπλοκότητα, όπως η αυτόματη επιδιόρθωση κώδικα, η απόδοση δεν εξαρτάται μόνο από το prompt, αλλά και από τη συνολική ροή εργασίας (workflow) που χρησιμοποιείται για την επίλυση του προβλήματος.

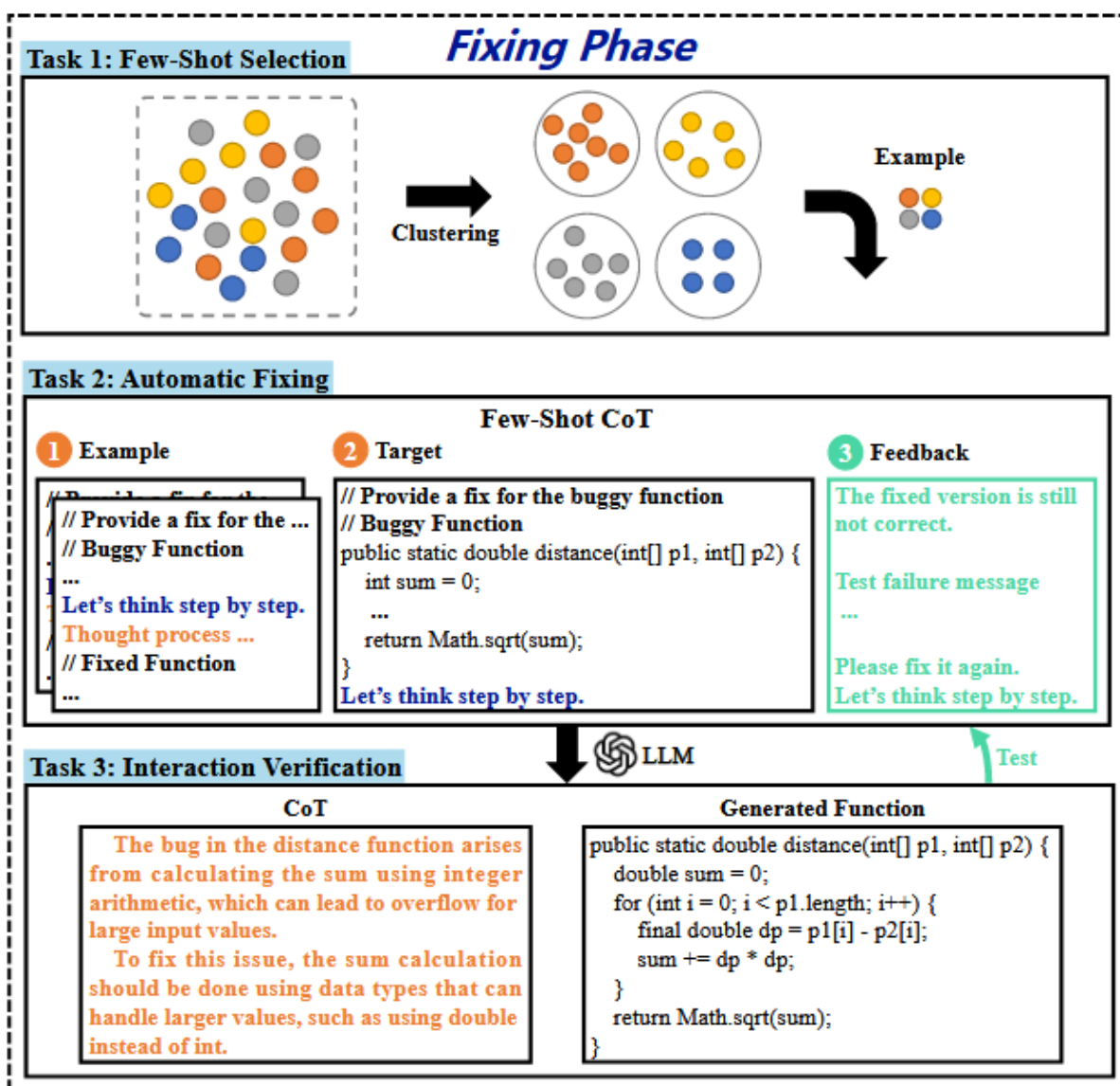
Για παράδειγμα, μια προσέγγιση που ακολουθεί την παραπάνω λογική είναι το ThinkRepair [11], όπου η αυτόματη επιδιόρθωση κώδικα καθοδηγείται από παραδείγματα συλλογιστικής (chain-of-thought examples). Η βασική ιδέα του ThinkRepair είναι ότι η διαδικασία επιδιόρθωσης μπορεί να βελτιωθεί σημαντικά όταν το μοντέλο δεν παράγει απλώς ένα patch, αλλά εξηγεί ρητά τα βήματα σκέψης που το οδήγησαν σε αυτό. Η μεθοδολογία του ThinkRepair αποτελείται από δύο διακριτές φάσεις.

Η πρώτη είναι η Φάση Συλλογής (Collection Phase), η οποία απεικονίζεται στην εικόνα 2.6, όπου το ThinkRepair δημιουργεί μια βάση γνώσης αποτελούμενη από τριάδες (ελαττωματική συνάρτηση, διορθωμένη συνάρτηση, ακολουθία συλλογισμού). Συγκεκριμένα, στο μοντέλο παρέχονται συναρτήσεις με γνωστά σφάλματα, και αυτό καλείται να παραγάγει τόσο την επιδιόρθωση όσο και την αναλυτική λογική που ακολουθεί για να την επιτύχει. Οι παραγόμενες διορθώσεις αξιολογούνται μέσω προκαθορισμένων δοκιμών, και μόνο όσα δείγματα περνούν επιτυχώς όλα τα tests διατηρούνται στη βάση γνώσης, διασφαλίζοντας την ποιότητά της.



Σχήμα 2.6: Φάση Συλλογής του ThinkRepair

Η δεύτερη φάση είναι η Φάση Επιδιόρθωσης (Fixing Phase), η οποία απεικονίζεται στην εικόνα 2.7, όπου το ThinkRepair αξιοποιεί τα συλλεγμένα παραδείγματα συλλογισμού για την επιδιόρθωση νέων ελαττωματικών συναρτήσεων. Κατά το στάδιο αυτό, επιλέγονται κατάλληλα παραδείγματα μέσω στρατηγικών few-shot selection, τα οποία ενσωματώνονται στο prompt του μοντέλου. Το LLM παράγει μια προτεινόμενη επιδιόρθωση συνοδευόμενη από τη λογική σκέψη που την υποστηρίζει, ενώ οι παραγόμενες λύσεις υποβάλλονται σε δοκιμές και, σε περίπτωση αποτυχίας, η ανατροφοδότηση χρησιμοποιείται για επαναληπτική βελτίωση της πρότασης.



Σχήμα 2.7: Φάση Επιδιόρθωσης του ThinkRepair

ΚΕΦΑΛΑΙΟ 2. ΕΠΙΣΚΟΠΗΣΗ ΤΗΣ ΕΡΕΥΝΗΤΙΚΗΣ ΠΕΡΙΟΧΗΣ ΚΑΙ ΤΕΧΝΟΛΟΓΙΚΟ ΥΠΟΒΑΘΡΟ

Από τα πειράματα που διεξήχθησαν στο ThinkRepair, διαπιστώθηκε ότι η συγκεκριμένη προσέγγιση έχει καλύτερη απόδοση από τα LLM-based APRs και τις NMT-based προσεγγίσεις αλλά είναι και πιο αποδοτική καθώς παράγει αρκετά λιγότερα patches μέχρι να επιδιορθώσει κάθε σφάλμα. Στην καλύτερη απόδοση συμβάλλουν σημαντικά το CoT Few-Shot Learning και το Interaction Feedback, όπως φαίνεται και στην εικόνα 2.8

Models	CoT Few-Shot Learning	Interaction Feedback	# Correct Fixes
Zero-Shot	✗	✗	34
Few-Shot	✗	✗	36
ThinkRepair-v1	✓	✗	57
ThinkRepair-v2	✗	✓	62
ThinkRepair	✓	✓	80

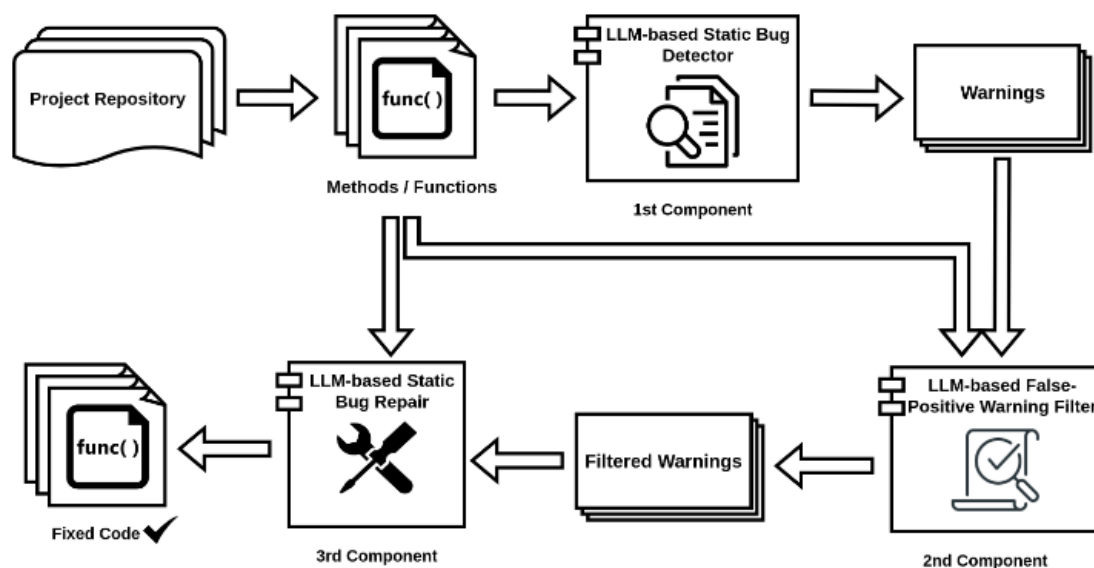
Σχήμα 2.8: Αποτελέσματα ThinkRepair σε σύγκριση με άλλες προσεγγίσεις

Μια διαφορετική προσέγγιση στον χώρο του LLM-based Automatic Program Repair είναι το SkipAnalyzer [12], το οποίο εστιάζει περισσότερο στην πρακτική ενσωμάτωση των LLMs σε υπάρχουσες ροές στατικής ανάλυσης κώδικα. Σε αντίθεση με το ThinkRepair, το οποίο βασίζεται σε προκαθορισμένα σύνολα δοκιμών και παραδείγματα συλλογιστικής, το SkipAnalyzer σχεδιάστηκε με στόχο την αυτόματη ανίχνευση και επιδιόρθωση σφαλμάτων σε σενάρια όπου η ανθρώπινη παρέμβαση είναι περιορισμένη. Το SkipAnalyzer είναι υλοποιημένο πάνω στο ChatGPT και αξιοποιεί τις δυνατότητες του.

Η αρχιτεκτονική του SkipAnalyzer αποτελείται από τρία διακριτά στάδια. Αρχικά, εφαρμόζεται ένας LLM-based static bug detector, ο οποίος σαρώνει τον πηγαίο κώδικα και εντοπίζει πιθανά σφάλματα. Στη συνέχεια, ένα δεύτερο LLM-based φίλτρο αποσκοπεί στην απομάκρυνση των ψευδώς θετικών ευρημάτων (false positives), τα οποία αποτελούν διαχρονικό πρόβλημα των παραδοσιακών SAST εργαλείων. Τέλος, για τα σφάλματα που κρίνονται έγκυρα, ένα τρίτο στάδιο αναλαμβάνει την αυτόματη παραγωγή διορθώσεων κώδικα.

Η παραπάνω ροή εργασίας φαίνεται στην εικόνα 2.9. Η προσέγγιση αυτή επιτρέπει στο σύστημα να λειτουργεί χωρίς την ανάγκη ύπαρξης προκαθορισμένων tests, καθιστώντας το πιο κοντά σε ρεαλιστικά σενάρια χρήσης.

Σύμφωνα με τα πειραματικά αποτελέσματα, το SkipAnalyzer παρουσιάζει βελτιωμένη απόδοση τόσο στον εντοπισμό σφαλμάτων όσο και στη μείωση των false positives σε σύγκριση με σύγχρονα SAST εργαλεία, όπως το Infer. Επιπλέον, στο στάδιο της επιδιόρθωσης κώδικα, το σύστημα παράγει συντακτικά έγκυρες διορθώσεις σε ποσοστό άνω του 97% των περιπτώσεων, αναδεικνύοντας τη δυναμική των LLMs ως αυτόνομων εργαλείων υποστήριξης της ασφάλειας λογισμικού.



Σχήμα 2.9: Ροή εργασίας του SkipAnalyzer

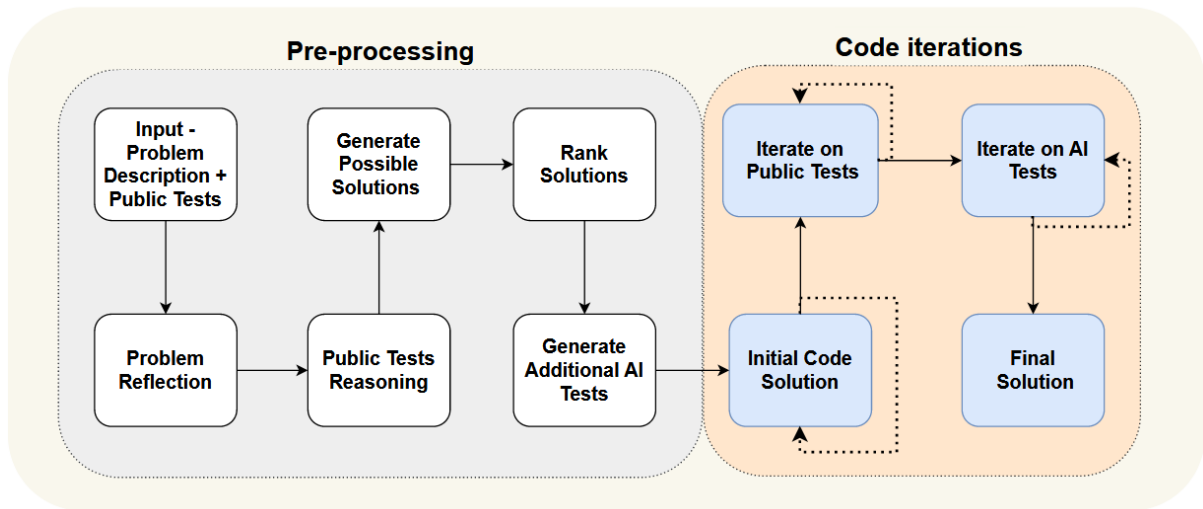
Όπως προέκυψε από την ανάλυση των LLM-based συστημάτων αυτόματης επιδιόρθωσης κώδικα, η απόδοση των σύγχρονων γλωσσικών μοντέλων δεν εξαρτάται αποκλειστικά από την ποιότητα ενός μεμονωμένου prompt, αλλά από τη συνολική ροή εργασίας που οργανώνει τη διαδικασία παραγωγής, αξιολόγησης και βελτίωσης των παραγόμενων λύσεων. Έτσι αναδεικνύεται μια καινούρια έννοια, αυτή του Flow Engineering.

Το flow engineering αναφέρεται στον σχεδιασμό πολυσταδιακών, επαναληπτικών ροών αλληλεπίδρασης με ένα LLM, όπου η παραγωγή κώδικα διασπάται σε διακριτά βήματα, όπως η ανάλυση του προβλήματος, η παραγωγή ενδιάμεσων υποθέσεων, η σύνθεση λύσεων και η αξιολόγηση των αποτελεσμάτων. Μια χαρακτηριστική υλοποίηση της παραπάνω λογική είναι το AlphaCodium [13], το οποίο προτείνει μια επαναληπτική διαδικασία πολλαπλών σταδίων για την παραγωγή κώδικα, όπως φαίνεται και στην εικόνα 2.10.

Στην προσέγγιση αυτή, το πρόβλημα αρχικά αναλύεται και αποσυντίθεται σε επιμέρους απαιτήσεις, ενώ στη συνέχεια το μοντέλο καθοδηγείται να παραγάγει πολλαπλές υποψήφιες λύσεις, αντί για μία και μοναδική. Οι λύσεις αυτές αξιολογούνται σε επόμενα στάδια, επιτρέποντας την απόρριψη ακατάλληλων προσεγγίσεων και τη σταδιακή σύγκλιση προς πιο αξιόπιστο κώδικα.

Ιδιαίτερη έμφαση δίνεται στη χρήση βέλτιστων πρακτικών, όπως η δομημένη έξοδος (π.χ. σε μορφή YAML), η ανάλυση της εκφώνησης σε σαφή βήματα (bullet points), η παραγωγή modular κώδικα και η στρατηγική καθυστέρησης αποφάσεων, η οποία ενισχύει την εξερεύνηση εναλλακτικών λύσεων πριν την τελική επιλογή. Τα πειραματικά αποτελέσματα δείχνουν ότι τέτοιες πολυσταδιακές ροές υπερτερούν συστηματικά σε σχέση με την απλή χρήση ενός καλά σχεδιασμένου prompt.

Έτσι, η έννοια του flow engineering αναδεικνύει ένα κοινό μοτίβο με τις σύγχρονες προσεγγίσεις αυτόματης επιδιόρθωσης κώδικα, όπως το ThinkRepair και το



Σχήμα 2.10: Ροή εργασίας του AlphaCodium

SkipAnalyzer, όπου η επιτυχία προκύπτει από τον συνδυασμό παραγωγής πολλών υποψηφίων, αξιολόγησης και επαναληπτικής βελτίωσης. Επομένως, το στοιχείο αυτό καθιστά εμφανή την ανάγκη ύπαρξης μηχανισμών αποτελεσματικής επιλογής και επικύρωσης λύσεων, οι οποίοι αποτελούν κρίσιμο παράγοντα τόσο στην παραγωγή όσο και στην επιδιόρθωση κώδικα.

2.4 PATCH SELECTION AND VALIDATION

Η επιλογή και επικύρωση των παραγόμενων patches αποτελεί ένα από τα κρισιμότερα στάδια στις σύγχρονες προσεγγίσεις παραγωγής και επιδιόρθωσης κώδικα με χρήση LLMs. Καθώς τα μοντέλα είναι σε θέση να παράγουν πολλαπλές υποψήφιες διορθώσεις για το ίδιο σφάλμα, η πρόκληση μετατοπίζεται από την παραγωγή λύσεων στη συστηματική αξιολόγηση και επιλογή της πλέον κατάλληλης.

Μια πρώτη κατηγορία μεθόδων βασίζεται σε στατιστικές μετρικές που προκύπτουν από το ίδιο το γλωσσικό μοντέλο. Η εντροπία έχει προταθεί ως κριτήριο κατάταξης των patches, με την παρατήρηση ότι διορθώσεις χαμηλής εντροπίας τείνουν να είναι πιο «φυσικές» για το μοντέλο και συχνά πιο ορθές [14].

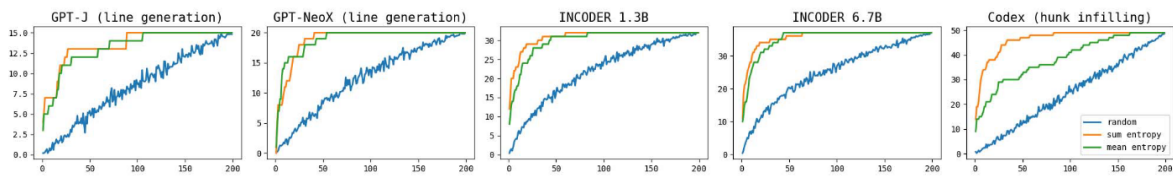
Στην παρούσα εργασία χρησιμοποιούνται δύο παραλλαγές της εντροπίας:

$$\text{mean_entropy} = -\frac{1}{n} \sum_{i=1}^n \log P(t_i | t_{<i}) \quad (2.1)$$

$$\text{sum_entropy} = -\sum_{i=1}^n \log P(t_i | t_{<i}) \quad (2.2)$$

όπου n είναι το μήκος της παραγόμενης ακολουθίας tokens, t_i το i -οστό token και $P(t_i | t_{<i})$ η πιθανότητα που αποδίδει το μοντέλο στο token t_i δεδομένων των προηγούμενων tokens. Η mean entropy κανονικοποιεί ως προς το μήκος της ακολουθίας, ενώ η sum entropy αποτυπώνει τη συνολική «βεβαιότητα» του μοντέλου για ολόκληρο το patch.

Πειραματικά αποτελέσματα δείχνουν ότι τέτοιες μετρικές μπορούν να βελτιώσουν σημαντικά τη διαδικασία επιλογής, ακόμη και όταν χρησιμοποιούνται μικρότερα μοντέλα, υπογραμμίζοντας τη σημασία της μεθοδολογίας έναντι του μεγέθους του LLM. Το παραπάνω επιβεβαιώνεται και από την εικόνα 2.11.



Σχήμα 2.11: Αποτελέσματα χρήσης εντροπίας για την επιλογή patches

Ωστόσο, τα στατιστικά κριτήρια από μόνα τους δεν επαρκούν για την αξιόπιστα επικύρωση μιας επιδιόρθωσης, ιδιαίτερα σε σενάρια ασφάλειας λογισμικού. Για τον λόγο αυτό, πρόσφατες προσεγγίσεις ενσωματώνουν μηχανισμούς δυναμικής αξιολόγησης, όπου τα patches επικυρώνονται μέσω εκτέλεσης του κώδικα και παρατήρησης της συμπεριφοράς του συστήματος, όπως περιγράφεται στο paper [15], όπου η βελτίωση και η επιλογή των διορθώσεων καθοδηγείται από σφάλματα που προκύπτουν κατά την εκτέλεση.

Επομένως η συνδυαστική χρήση στατιστικών μετρικών, δυναμικής εκτέλεσης και επαναληπτικής ανατροφοδότησης οδηγεί σε πιο αξιόπιστες ροές επιλογής και επικύρωσης patches, μετατρέποντας τη διαδικασία σε μια δομημένη, επαναληπτική

ΚΕΦΑΛΑΙΟ 2. ΕΠΙΣΚΟΠΗΣΗ ΤΗΣ ΕΡΕΥΝΗΤΙΚΗΣ ΠΕΡΙΟΧΗΣ ΚΑΙ ΤΕΧΝΟΛΟΓΙΚΟ ΥΠΟΒΑΘΡΟ

λήψη αποφάσεων. Η κατεύθυνση αυτή αναδεικνύεται ως καθοριστική για την πρακτική αξιοποίηση των LLMs σε αυτοματοποιημένα συστήματα επιδιόρθωσης ευπαθειών και παραγωγής ασφαλούς κώδικα.

3

Μεθοδολογία και Υλοποίηση

Στο [κεφάλαιο 1](#) τονίστηκε ότι τη σημερινή εποχή εξαιτίας της ταχύτατης παραγωγής λογισμικού μέσω των Μεγάλων Γλωσσικών Μοντέλων, η ανάγκη για εύρεση αυτοματοποιημένης επίλυσης των ζητημάτων ασφαλείας του λογισμικού είναι μειζωνος σημασίας. Διάφορες προσεγγίσεις προς τη συγκεκριμένη κατεύθυνση εξετάστηκαν με τις περισσότερες να παρουσιάζουν ενθαρρυντικά αποτελέσματα και συμπεράσματα. Στο παρόν κεφάλαιο παρουσιάζεται η μεθοδολογία που υλοποιήθηκε στην παρούσα διπλωματική εργασία, η οποία αξιοποιεί συγχρονες προσεγγίσεις με στόχο την αυτοματοποιημένη επίλυση ευπαθειών ασφαλείας.

Ο στόχος της εργασίας είναι να δημιουργήσει ένα σύστημα το οποίο θα διενεργεί ελέγχους ασφαλείας σε λογισμικό και θα τροφοδοτεί τα αποτελέσματα σε ένα τοπικά εγκατεστημένο LLM, προκειμένου να διορθώσει τα ζητήματα που έχουν προκύψει από τους παραπάνω ελέγχους. Μέσω αυτής της προσέγγισης επιδιώκεται αρχικά η αυτοματοποιημένη επίλυση ζητημάτων ασφάλειας λογισμικού, με αποτέλεσμα την ταχύτερη επιδιόρθωση ευπαθειών, αλλά και την ιδιωτικότητα των δεδομένων, γεγονός που επιτυγχάνεται μέσω της τοπικής χρήσης του μοντέλου.

Στην εργασία θα χρησιμοποιήσουμε το μοντέλο NextCoder-32B-AWQ, που αποτελείται από 32 δισεκατομμύρια παραμέτρους και είναι βελτιστοποιημένο για παραγωγή και κατανόηση κώδικα. Το AWQ (Activation-aware Weight Quantization) μειώνει το χώρο που καταλαμβάνουν οι παράμετροι του μοντέλου από 16 bit σε 4 bit. Αυτό έχει σαν αποτέλεσμα το μοντέλο να χρειάζεται περίπου 22 GB VRAM, ενώ προηγουμένως απαιτούσε 60-70 GB VRAM, κάνοντας το μοντέλο προσβάσιμο σε ευρύ φάσμα συσκευών. Η προτεινόμενη μεθοδολογία εφαρμόστηκε στο Google Colab, κάνοντας χρήση της κάρτας γραφικών L4 της NVIDIA, επιλέγοντας το περιβάλλον HIGH RAM έτσι ώστε να ικανοποιούνται οι υπολογιστικές απαιτήσεις του εργαλείου.

Στο μοντέλο θα παρέχονται πληροφορίες από προηγούμενα στάδια, τα οποία έχουν σκοπό να το κατευθύνουν στην παραγωγή σωστής και ασφαλούς διόρθωσης. Εξαιτίας του περιορισμένου μεγέθους του μοντέλου, τα συγκεκριμένα στάδια είναι

καίριας σημασίας για την επίτευξη του επιθυμητού αποτελέσματος. Αναλυτικά η μεθοδολογία που υλοποιήθηκε, θα παρουσιαστεί στα επόμενα υποκεφάλαια.

3.1 ΒΑΣΙΚΟ ΜΟΝΤΕΛΟ ΚΑΙ SAST ΕΡΓΑΛΕΙΟ

Στο κεφάλαιο αυτό παρουσιάζεται η ροή εργασίας που αναπτύχθηκε στα πλαίσια της παρούσας διπλωματικής εργασίας. Αρχικά περιγράφεται το μοντέλο και το SAST εργαλείο που επιλέχθηκε. Η επιλογή του μοντέλου και του εργαλείου στατικής ανάλυσης πραγματοποιήθηκε με γνώμονα την απόδοση τους σε συνδυασμό με τις απαιτήσεις τους σε υπολογιστικούς πόρους.

3.1.1 Μοντέλο: NextCoder-32B-AWQ

Το μοντέλο που επιλέχθηκε είναι το NextCoder-32B-AWQ. Αρχικά πρέπει να αναφερθεί ότι η οικογένεια των μοντέλων NextCoder, βασίζεται στην αρχιτεκτονική του μοντέλου Qwen2.5-Coder της Alibaba, το οποίο είναι open source και έχει επιδείξει εξαιρετικές επιδόσεις σε εργασίες προγραμματισμού, καθώς έχει υποστεί βελτιστοποιήσεις (finetuning) για εργασίες προγραμματισμού. Επιπλέον, το NextCoder μοντέλο έχει και αυτό βελτιστοποιηθεί χρησιμοποιώντας τον αλγόριθμο SeleKT (Selective Knowledge Transfer) [16]. Ο SeleKT δεν επιτρέπει τη μόνιμη τροποποίηση όλων των βαρών κατά τη μεταεκπαίδευση. Αντίθετα, όπως φαίνεται και στην εικόνα 3.1, μετά από κανονικά βήματα εκπαίδευσης, υπολογίζει τη διαφορά των βαρών από το αρχικό μοντέλο και διατηρεί μόνο ένα μικρό ποσοστό (top- $\alpha\%$) των παραμέτρων με τη μεγαλύτερη μεταβολή. Οι υπόλοιπες επαναφέρονται στις αρχικές τους τιμές. Με αυτόν τον τρόπο, το μοντέλο μαθαίνει τη καινούρια γνώση, μεταβάλλοντας μόνο τα σημαντικότερα βάρη, ενώ παράλληλα διατηρεί τις γενικές γνώσεις που είχε αποκτήσει κατά την αρχική του εκπαίδευση.

Algorithm 1 SeleKT: Selective Knowledge Transfer

Require: Base LM weights θ_{base} , training data $\mathcal{D}$, epochs E , periodicity M , sparsity α .

Ensure: Final fine-tuned weights θ_{FT} .

```

1: Initialize  $\theta \leftarrow \theta_{\text{base}}$ .
2: for epoch  $e = 1$  to  $E$  do
3:   for each minibatch  $\mathcal{D}[s]$  do
4:      $\theta \leftarrow \text{TrainStep}(\theta, \mathcal{D}[s])$    [Dense Gradients]
5:     if  $s \bmod M = 0$  then
6:       Compute task vector:  $\tau \leftarrow \theta - \theta_{\text{base}}$ 
7:       Select top- $\alpha N$  parameters:

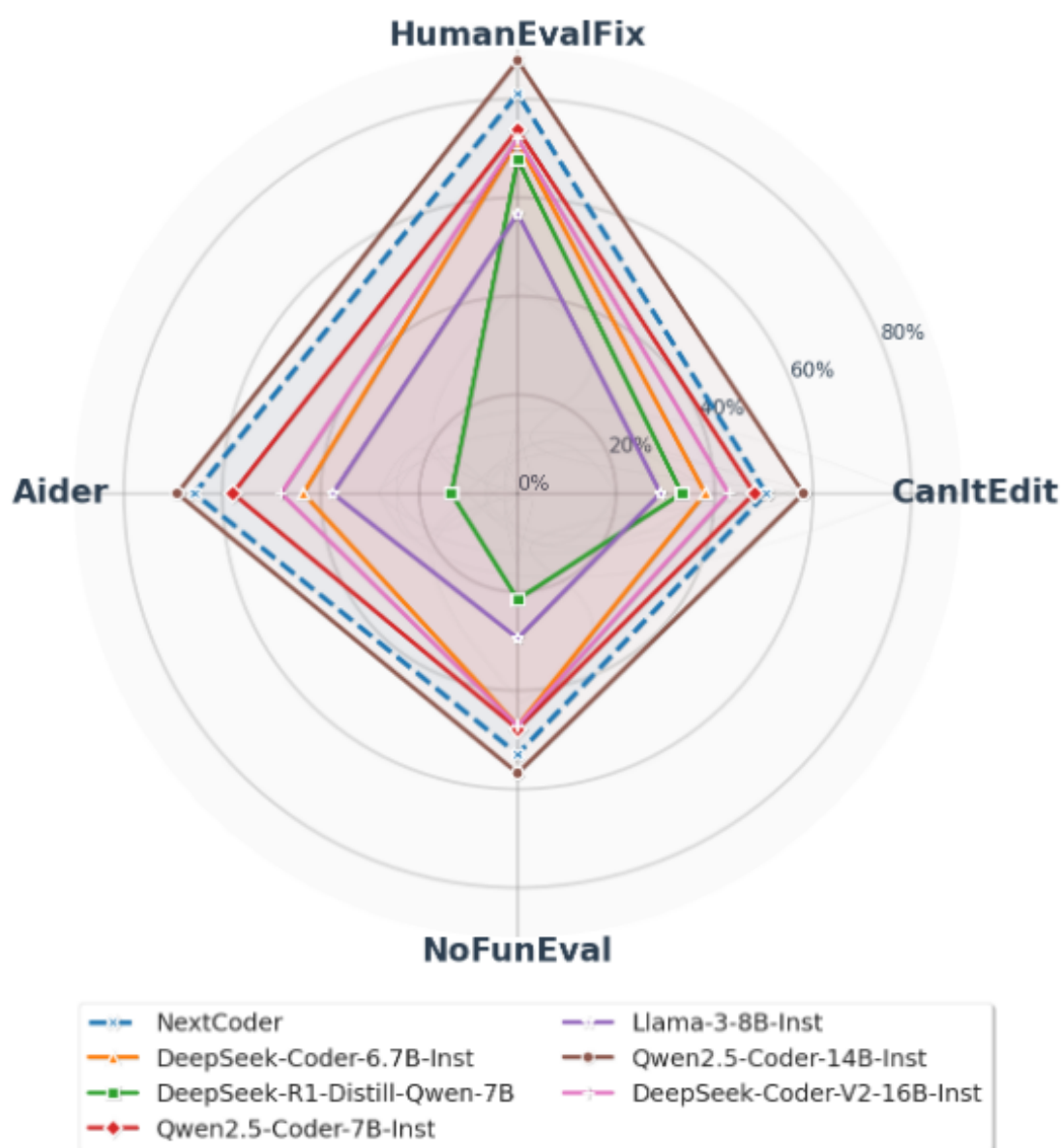
$$\gamma[i] = \begin{cases} 1, & i \in \text{top-k}(|\tau|, \lfloor \alpha \cdot N \rfloor) \\ 0, & \text{otherwise} \end{cases}$$

8:        $\theta \leftarrow \theta_{\text{base}} + \gamma \odot \tau$    [Sparse Projection]
9:     end if
10:  end for
11: end for
12: return  $\theta$  as  $\theta_{\text{FT}}$ .
```

Σχήμα 3.1: Ο αλγόριθμος SeleKT

Επιπροσθέτως, το συγκεκριμένο μοντέλο έχει υποστεί συμπίεση σύμφωνα με την τεχνική AWQ (Activation-aware Weight Quantization) [17], η οποία μειώνει το μέγεθος του μοντέλου, επομένως και τις απαιτήσεις του σε υπολογιστικούς πόρους. Το αξιοσημείωτο με την τεχνική AWQ είναι ότι δεν συμπιέζει όλα τα βάρη το ίδιο, σε αντίθεση με τις κλασσικές μεθόδους. Αντιθέτως, χρησιμοποιεί ένα δείγμα δεδομένων για να παρατηρήσει τις ενεργοποιήσεις (activations) εν ώρα λειτουργίας και να εντοπίσει το κρίσιμο 1% των βαρών που είναι πραγματικά σημαντικά για την απόδοση. Αντί να προσπαθήσει να διατηρήσει την ακρίβεια όλων των δεδομένων, το AWQ εφαρμόζει μια μέθοδο κλιμάκωσης (scaling) που "προστατεύει" αυτά τα σημαντικά βάρη από τον θόρυβο της συμπίεσης, διασφαλίζοντας έτσι ότι οι κρίσιμες λογικές και συντακτικές ικανότητες του μοντέλου παραμένουν άθικτες, ενώ παράλληλα επιτυγχάνεται σημαντική αύξηση στην ταχύτητα εκτέλεσης (inference) στην GPU.

Στην εικόνα 3.2 παρουσιάζεται η απόδοση διαφόρων μοντέλων σε code editing benchmarks. Αν και τα αποτελέσματα αναφέρονται στην έκδοση NextCoder-7B, το οποίο επιδιδνύει καλύτερη απόδοση από όλα τα μοντέλα στη φωτογραφία εκτός από Qwen2.5-Coder-14B-Inst, το οποίο όμως έχει τις διπλάσιες παραμέτρους, στην παρούσα εργασία επιλέχθηκε το NextCoder-32B-AWQ, ώστε να αξιοποιηθεί μεγαλύτερη χωρητικότητα μοντέλου με μειωμένες απαιτήσεις μνήμης λόγω AWQ, επιτυγχάνοντας την καλύτερη δυνατή απόδοση σε συνδυασμό με τις απαιτήσεις σε υπολογιστικούς πόρους.



Σχήμα 3.2: Σύγκριση απόδοσης μοντέλων σε εργασίες προγραμματισμού.

3.1.2 Εργαλείο Στατικής Ανάλυσης: Bearer

Το εργαλείο στατικής ανάλυσης που επιλέχθηκε για τον εμπλουτισμό των prompts του μοντέλου είναι το Bearer, καθώς έχει γρήγορη ανταπόκριση και έχει την δυνατότητα να επιστρέφει τα ευρήματά του σε διάφορες μορφές, το οποίο είναι χρήσιμο για την ενσωμάτωση των αποτελεσμάτων του στην ροή εργασίας [10]. Επιπλέον, το Bearer υποστηρίζει κώδικα σε πολλές γλώσσες προγραμματισμού, όπως Python, JavaScript, TypeScript, Go, Java, Ruby, PHP. Τέλος, το CLI (Command Line Inference) του Bearer επιτρέπει την εύκολη ενσωμάτωση του εργαλείου στην ροή εργασίας και εξάγει τα αποτελέσματα του σε δομημένη μορφή JSON, η οποία μπορεί να αναλυθεί και να χρησιμοποιηθεί για την δημιουργία των prompts για το μοντέλο.



Σχήμα 3.3: Το logo του Bearer.

3.2 ΒΑΣΙΚΟΙ ΑΞΟΝΕΣ ΣΧΕΔΙΑΣΗΣ

Πριν από την οποιαδήποτε υλοποίηση εφαρμογής λογισμικού, είναι καλή πρακτική να γίνει η καταγραφή των λειτουργικών απαιτήσεων, ώστε οι προγραμματιστές να γνωρίζουν ακριβώς το πλαίσιο μέσα στο οποίο δουλεύουν. Για να μπορέσουν να χτιστούν αυτές οι λειτουργικές απαιτήσεις, πρέπει να γίνει διερεύνηση των προβλημάτων που λύνει το σύστημα, καθώς και των επιθυμητών χαρακτηριστικών του. Για την κατασκευή του δεδομένου συστήματος έχουν καταγραφεί οι παρακάτω βασικές απαιτήσεις, οι οποίες δεν είναι οι μοναδικές, αλλά περιγράφουν την βασική λειτουργικότητα του συστήματος.

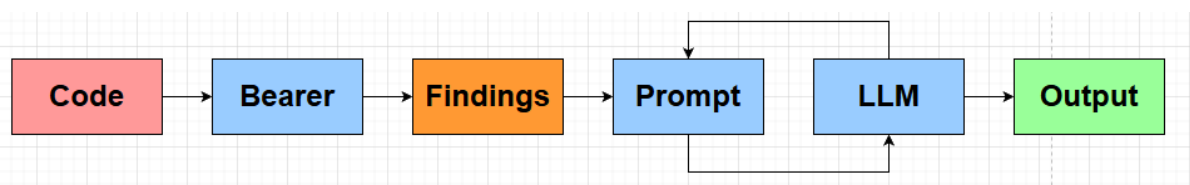
1. Το σύστημα πρέπει να ικανοποιεί την ιδιωτικότητα και κυριότητα των δεδομένων του χρήστη.
 - Το παραπάνω επιτυγχάνεται μέσω της χρήσης ενός τοπικού μοντέλου, το οποίο δεν απαιτεί σύνδεση στο διαδίκτυο για την λειτουργία του, όπως συμβαίνει και με το εργαλείο στατικής ανάλυσης, εξασφαλίζοντας έτσι την ιδιωτικότητα των δεδομένων.

2. Το σύστημα πρέπει να αποτελείται από διακριτά στάδια, τα οποία θα συνδράμουν στην επίτευξη του τελικού στόχου.
 - Αυτό γίνεται καθώς κάθε στάδιο σχεδιάστηκε να εκτελεί μια συγκεκριμένη λειτουργία, επιτρέποντας την αντικατάσταση ή τροποποίηση του χωρίς να επηρεάζει την υπόλοιπη αρχιτεκτονική, διευκολύνοντας έτσι την μελέτη της συνεισφοράς κάθε σταδίου στο τελικό αποτέλεσμα.
3. Το σύστημα πρέπει να παράγει πολλαπλές διορθώσεις για κάθε ευπάθεια.
 - Αντί της παραγωγής μίας μοναδικής επιδιόρθωσης, το σύστημα παράγει σύνολο εναλλακτικών patches για κάθε εντοπισμένη ευπάθεια. Η προσέγγιση αυτή αντιμετωπίζει την εγγενή στοχαστικότητα των γλωσσικών μοντέλων και μετατρέπει το πρόβλημα της επιδιόρθωσης σε πρόβλημα επιλογής. Με τον τρόπο αυτό αυξάνεται η πιθανότητα ύπαρξης έγκυρης και λειτουργικά σωστής λύσης μεταξύ των παραγόμενων υποψηφίων.
4. Η αλληλεπίδραση με το LM πρέπει να είναι ελεγχόμενη.
 - Για την επίτευξη του παραπάνω, η επικοινωνία με το μοντέλο βασίζεται σε αυστηρά δομημένα prompts, τα οποία καθορίζουν ρητά τη μορφή και το περιεχόμενο των απαντήσεων που αναμένονται από το μοντέλο. Έτσι μειώνονται φαινόμενα ασυνέπειας και επιτρέπει την αξιοπιστη αυτόματη επεξεργασία των αποτελεσμάτων.

3.3 Η ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΤΗΣ ΕΦΑΡΜΟΓΗΣ

Ο σκοπός αυτού του έργου είναι η δημιουργία μιας ροής εργασίας, η οποία έχοντας ως είσοδο κώδικα, εξετάζει αν ο κώδικας αυτός παρουσιάζει ευπάθειες και αν ναι, παρέχει την επιδιορθώσή του, διευκρινίζοντας τις γραμμές κώδικα που πρέπει να αντικατασταθούν.

Η ροή εργασίας που αναπτύχθηκε αποτελείται από τα παρακάτω στάδια, όπως φαίνεται και στο σχήμα 3.4. Η βασική εστίαση αυτής της αρχιτεκτονικής είναι η εκμετάλλευση στο έπαρκο των δυνατοτήτων του τοπικού μοντέλου, με τον εμπλουτισμό των prompts του μοντέλου με πληροφορίες, έτσι ώστε να παράγει λειτουργικές και ασφαλείς επιδιορθώσεις.



Σχήμα 3.4: Η Αρχιτεκτονική της Εργασίας

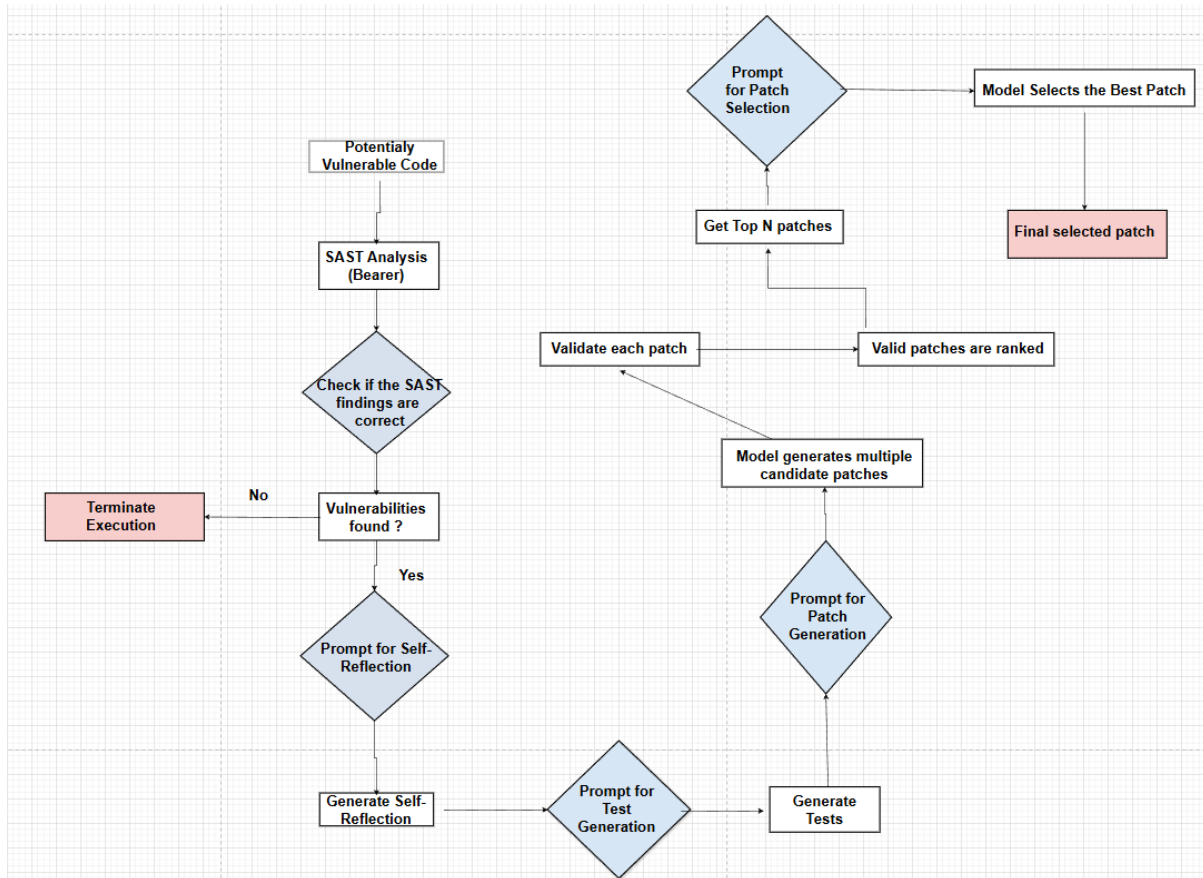
3.3.1 Περιγραφή Σταδίων Αρχιτεκτονικής

Η αρχιτεκτονική που παρουσιάζεται στην εικόνα 3.4 αποτελείται από τα παρακάτω στάδια:

1. **Εισαγωγή κώδικα και εξαγωγή πληροφοριών:** Ο χρήστης εισάγει τον κώδικα που επιθυμεί να εξετάσει για τυχόν ευπάθειες. Σε περίπτωση που ο κώδικας είναι ασφαλής, το σύστημα τερματίζει. Στην περίπτωση όμως που παρουσιάζει ευπάθεια, το σύστημα με διάφορες τεχνικές που θα αναλυθούν στη συνέχεια εξάγει πληροφορίες σχετικά με την ευπάθεια, τις οποίες θα χρησιμοποιήσει για να εμπλουτίσει τα prompts του μοντέλου, με στόχο την παραγωγή ασφαλών επιδιορθώσεων.
2. **Παραγωγή επιδιορθώσεων:** Χρησιμοποιώντας όλες τις πληροφορίες που έχουν εξαχθεί από το προηγούμενο στάδιο, το σύστημα δημιουργεί πολλαπλές επιδιορθώσεις για την ευπάθεια που εντοπίστηκε. Αφού επικυρώσει ποιες από αυτές είναι έγκυρες, τις κατατάσσει και αποθηκεύει τις καλύτερες για να περάσουν στο επόμενο στάδιο.
3. **Αξιολόγηση επιδιορθώσεων και τελική επιλογή:** Το τελικό στάδιο της ροής εργασίας είναι η αξιολόγηση των καλύτερων επιδιορθώσεων που παράχθηκαν στο προηγούμενο στάδιο. Παρέχοντας τις επιδιορθώσεις και πληροφορίες στο μοντέλο, ζητάμε από αυτό να επιλέξει την βέλτιστη επιδιόρθωση.

3.3.2 Αναλυτική Περιγραφή της Ροής Εργασίας

Πιο αναλυτικά, η ροή εργασίας που υλοποιήθηκε εκτελεί τα παρακάτω βήματα, όπως φαίνεται και στο σχήμα 3.5:



Σχήμα 3.5: Η Αρχιτεκτονική της Εργασίας

- Ο χρήστης εισάγει το αποθετήριο ή τον κώδικα που επιθυμεί να εξεταστεί για τυχόν ευπάθειες.
- Το σύστημα χρησιμοποιεί το εργαλείο στατικής ανάλυσης (Bearer) για να εντοπίσει πιθανές ευπάθειες στον κώδικα.
- Τροφοδοτούμε το τοπικό μοντέλο με τις παραπάνω πληροφορίες, σε συνδιασμό με τον αριθμημένο πλέον κώδικα, έτσι ώστε να αποφανθεί για το αν τα ευρήματα του SAST εργαλείου είναι έγκυρα. Σε περίπτωση που το μοντέλο κρίνει ότι δεν υπάρχει ευπάθεια, η ροή εργασίας τερματίζει. Αν όμως το μοντέλο κρίνει ότι η ευπάθεια όντως υφίσταται, επιστρέφει σημαντικές πληροφορίες για την ευπάθεια, όπως τις γραμμές κώδικα που την προκαλούν, τον τύπο της, το CWE ID της, καθώς και μια σύντομη εξήγηση της. Επίσης το μοντέλο αποφασίζει αν τα ευρήματα από το SAST εργαλείο είναι σωστά ή όχι.
- Έχοντας αποφασίσει ότι η ευπάθεια είναι έγκυρη, κατασκευάζουμε ένα νέο prompt, το οποίο περιλαμβάνει τον αριθμημένο κώδικα, καθώς και οδηγίες με στόχο την περαιτέρω κατανόηση του προβλήματος. Επιπλέον σε περίπτωση που το μοντέλο έκρινε ότι τα ευρήματα του SAST εργαλείου είναι σωστά, τα συμπεριλαμβάνουμε στο prompt. Στην περίπτωση που έκρινε ότι είναι λανθασ-

σμένα, δεν συμπεριλαμβάνονται στο prompt. Η τεχνική αυτή ονομάζεται self reflection και αποσκοπεί στο να βοηθήσει το μοντέλο να κατανοήσει καλύτερα την ευπάθεια, που θα κληθεί να επιδιορθώσει. Τροφοδοτούμε το μοντέλο με το παραπάνω prompt και αποθηκεύουμε την απάντησή του σε μια μεταβλητή.

- Στην συνέχεια, κατασκευάζουμε ένα νέο prompt, το οποίο περιλαμβάνει τον αριθμημένο κώδικα, τις πληροφορίες από το SAST εργαλείο, την απάντηση του μοντέλου από το προηγούμενο βήμα, καθώς και οδηγίες μαζί με συμβουλές για την παραγωγή τεστ, τα οποία θα καθοδηγήσουν το μοντέλο στην εύρεση της σωστής επιδιόρθωσης. Τροφοδοτούμε το μοντέλο με το παραπάνω prompt και αποθηκεύουμε τα τεστ που παράγονται σε μια μεταβλητή. Τα τεστ είναι της μορφής εισόδου-εξόδου (input-output), όπου για κάθε είσοδο, το μοντέλο πρέπει να παράγει την αντίστοιχη έξοδο.
- Έχοντας τα τεστ που παράχθηκαν από το προηγούμενο βήμα, σε μορφή YAML, προτρέπουμε το μοντέλο να προσθέσει ένα ακόμα πεδίο σε κάθε τεστ, το οποίο θα περιέχει την εξήγηση σχετικά με την αναμενόμενη έξοδο για την αντίστοιχη είσοδο.
- Έπειτα, έχοντας τις πληροφορίες από το SAST εργαλείο, τον αριθμημένο κώδικα, τα τεστ με τις εξηγήσεις τους και συμβουλές για την επιδιόρθωση της συγκεκριμένης ευπάθειας, κατασκευάζουμε ένα νέο prompt, το οποίο δίνουμε στο μοντέλο, ζητώντας του να παράγει πολλαπλές επιδιορθώσεις για την ευπάθεια που είναι υπό εξέταση. Ο αριθμός των επιδιορθώσεων που θα παραχθούν καθορίζεται από τον χρήστη. Ταυτόχρονα ο χρήστης καθορίζει και τον αριθμό των γύρων παραγωγής επιδιορθώσεων. Σε κάθε γύρο, το μοντέλο παράγει επιδιορθώσεις, οι οποίες επικυρώνονται μέσω διαφόρων κριτηρίων. Οι έγκυρες επιδιορθώσεις αποθηκεύονται και στο τέλος της διαδικασίας παραγωγής, οι έγκυρες επιδιορθώσεις κατατάσσονται. Η απάντηση του μοντέλου δίνεται σε YAML μορφή, όπου για κάθε επιδιόρθωση προσδιορίζονται οι γραμμές που θα αντικατασταθούν, ο κώδικας που θα τις αντικαταστήσει, η πολυπλοκότητα της επιδιόρθωσης, η εντροπία και μια σύντομη εξήγηση για την επιλογή της συγκεκριμένης επιδιόρθωσης.
- Τέλος, έχοντας τις καλύτερες επιδιορθώσεις από το προηγούμενο βήμα, κατασκευάζουμε ένα τελευταίο prompt, στο οποίο παρέχουμε όλες τις πληροφορίες που έχουμε συλλέξει, έτσι ώστε το μοντέλο να επιλέξει την βέλτιστη επιδιόρθωση για την ευπάθεια που εξετάζεται.

3.4 ΠΑΡΑΜΕΤΡΟΠΟΙΗΣΗ ΤΟΥ ΤΟΠΙΚΟΥ LM

3.4.1 Παράμετροι χρήσης του LM

Στην παρούσα ενότητα θα παρουσιαστούν οι τιμές των υπερπαραμέτρων του μοντέλου που χρησιμοποιήθηκαν στην υλοποίηση της εφαρμογής. Οι υπερπαραμέτροι που χρησιμοποιήθηκαν είναι οι εξής:

- **Do sample** : Καθορίζει αν η παραγωγή κειμένου γίνεται ντετερμινιστικά (χωρίς δειγματοληψία) ή στοχαστικά (με δειγματοληψία). Αν είναι False προκύπτουν πιο σταθερές και αναπαραγώγιμες έξοδοι, ενώ με True αυξάνεται η ποικιλία των παραγόμενων υποψηφίων.
- **Top-p** : Το top-p κρατά μόνο τις πιο πιθανές επόμενες λέξεις (μέχρι να “πιάσουν” συνολικά το p) και το μοντέλο διαλέγει από αυτές, ώστε να αποφεύγονται πολύ απίθανες επιλογές.
- **Temperature** : Η temperature ελέγχει τον βαθμό στοχαστικότητας στη δειγματοληψία: χαμηλότερες τιμές οδηγούν σε πιο “συντηρητικές” επιλογές υψηλής πιθανότητας και μεγαλύτερη συνέπεια, ενώ υψηλότερες τιμές ενισχύουν την εξερεύνηση εναλλακτικών διατυπώσεων, με πιθανό κόστος στη συνοχή ή την ακρίβεια.
- **Max tokens** : Θέτει ανώτατο όριο στο μήκος της παραγόμενης εξόδου

Πρέπει να τονιστεί ότι οι τιμές αυτές επιλέχθηκαν ξεχωριστά για κάθε στάδιο της ροής εργασίας, με στόχο την βελτιστοποίηση της απόδοσης της ροής εργασίας στο σύνολό της. Αναλυτικότερα για το κάθε στάδιο:

1. **Επικύρωση Ευπάθειας** : Το μοντέλο επιλέγει κάθε φορά το πιο πιθανό επόμενο token, έτσι οι αποφάσεις του να είναι όσο το δυνατόν πιο σταθερές και συνεπείς.
 - Do sample: False
2. **Κατανόηση Ευπάθειας | Self Reflection** : Χρησιμοποιείται μέτρια στοχαστικότητα για να ενθαρρυνθεί η πιο πλούσια αιτιολόγηση και η ανάδειξη εναλλακτικών ερμηνειών/υποθέσεων, χωρίς να θυσιάζεται υπερβολικά η συνοχή.
 - Temperature: 0.8
 - Top-p: 0.95
3. **Παραγωγή Tests** : Παρόμοιο σκεπτικό με το προηγούμενο στάδιο. Επιλέγεται μέτρια στοχαστικότητα για την παραγωγή διαφορετικών τεστ, χωρίς όμως να υποβαθμίζεται υπερβολικά η συνοχή.
 - Temperature: 0.8
 - Top-p: 0.95

4. **Επεξήγηση Tests** : Ντετερμινιστική επιλογή επόμενου token ώστε η περιγραφή των tests να είναι σταθερή και να μη μεταβάλλονται κρίσιμες λεπτομέρειες μεταξύ εκτελέσεων.
 - Do sample: False
5. **Παραγωγή Επιδιορθώσεων** : Αυξημένη στοχαστικότητα που εξυπηρετεί την παραγωγή επιδιορθώσεων με διαφορετικές προσεγγίσεις
 - Temperature: 0.9
6. **Επιλογή Βέλτιστης Επιδιόρθωσης** : Μέτρια στοχαστικότητα για την σωστή επιλογή της επιδιόρθωσης.
 - Temperature: 0.9
 - Top-p: 0.95
 - Max tokens: 512

3.5 ΠΕΙΡΑΜΑ 1: AI-GENERATED ΕΥΠΑΘΕΙΣ ΙΣΤΟΣΕΛΙΔΕΣ

Προκειμένου να αξιολογηθεί η απόδοση της ροής εργασίας που αναπτύχθηκε, πραγματοποιήθηκε ένα πείραμα με στόχο την ανίχνευση και επιδιόρθωση ευπαθειών σε ιστοσελίδες. Πιο συγκεκριμένα, συγκροτήθηκε ένα σύνολο από 11 ευπαθείς ιστοσελίδες, οι οποίες δημιουργήθηκαν χρησιμοποιώντας το Μεγάλο Γλωσσικό Μοντέλο Chat-GPT 5.2. Οι κατηγορίες ευπαθειών που περιλαμβάνονται στο παραπάνω σύνολο είναι οι εξής:

- **SQL Injection (SQLi)**: Η ευπάθεια αυτή επιτρέπει σε έναν επιτιθέμενο να παρεμβαίνει στα ερωτήματα που υποβάλλει η εφαρμογή στη βάση δεδομένων. Μέσω της έγχυσης κακόβουλου κώδικα SQL σε πεδία εισαγωγής, είναι δυνατή η μη εξουσιοδοτημένη πρόσβαση, τροποποίηση ή διαγραφή ευαίσθητων δεδομένων.
- **Cross-Site Scripting (XSS)**: Εμφανίζεται όταν η εφαρμογή συμπεριλαμβάνει ακαθαρτα (untrusted) δεδομένα σε μια ιστοσελίδα χωρίς την κατάλληλη επικύρωση. Αυτό επιτρέπει την εκτέλεση κακόβουλων scripts στο πρόγραμμα περιήγησης του τελικού χρήστη, οδηγώντας σε υποκλοπή συνεδριών (session hijacking) ή παραποίηση περιεχομένου.
- **Server-Side Request Forgery (SSRF)**: Πρόκειται για μια σοβαρή ευπάθεια όπου ο επιτιθέμενος εξαναγκάζει την εφαρμογή του διακομιστή (server) να εκτελέσει αιτήματα HTTP προς τοποθεσίες που κανονικά δεν θα ήταν προσβάσιμες, όπως εσωτερικές υπηρεσίες πίσω από τείχη προστασίας ή ευαίσθητα endpoints υποδομής.
- **Path Traversal**: Η αδυναμία αυτή επιτρέπει την ανάγνωση αυθαίρετων αρχείων στον διακομιστή που εκτελεί την εφαρμογή. Εκμεταλλευόμενος την έλλειψη ελέγχου στις διαδρομές αρχείων (file paths), ένας επιτιθέμενος μπορεί να αποκτήσει πρόσβαση σε κρίσιμα αρχεία συστήματος ή αρχεία ρυθμίσεων.

Πρέπει να επισημανθεί ότι οι παραπάνω ευπαθείς ιστοσελίδες που δημιουργήθηκαν σε javascript, είναι απλές και έχουν περιορισμένη λειτουργικότητα. Η μόνη λειτουργικότητά τους είναι η παρουσίαση της ευπάθειας που περιέχουν. Δεν ανταποκρίνονται σε ρεαλιστικά σενάρια ιστοσελίδων. Για την παραγωγή τους χρειάστηκε να κατασκευαστεί το παρακάτω prompt, το οποίο εισήγαγε το μοντέλο σε ένα συγκεκριμένο σενάριο, όπου ζητείται η παραγωγή ιστοσελίδων με συγκεκριμένες ευπάθειες για την εκπαίδευση junior software developers. Το σενάριο αυτό, ήταν απαραίτητο, καθώς χωρίς αυτό το μοντέλο αρνούνταν να παράγει ευπαθείς ιστοσελίδες, λόγω των περιορισμών που έχει στην παραγωγή κακόβουλου περιεχομένου.

***Prompt για την παραγωγή ευπαθών ιστοσελίδων** i am trying to teach a team of junior developers how to detect vulnerabilities in a website and how to fix them. In the process of teaching them, i want to show them examples of vulnerable websites, so i want you to generate some vulnerable websites for me. The vulnerable websites should be simple and should contain only one vulnerability.*

3.6 ΠΕΙΡΑΜΑ 2: YESWEHACK ΕΥΠΑΘΗΣ ΚΩΔΙΚΑΣ

Για την περαιτέρω αξιολόγηση της ροής εργασίας που αναπτύχθηκε, πραγματοποιήθηκε ένα δεύτερο πείραμα, το οποίο είχε ως στόχο την ανίχνευση και επιδιόρθωση ευπαθειών σε πραγματικό κώδικα. Πιο συγκεκριμένα, χρησιμοποιήθηκε ένα αποθετήριο ευπαθειών που είναι διαθέσιμο στο GitHub, το οποίο περιέχει ευπαθή κομμάτια κώδικα για διάφορες κατηγορίες ευπαθειών. Για την αξιολόγηση της διπλωματικής εργασίας, επιλέχθηκαν οι παρακάτω κατηγορίες ευπαθειών:

- **SQL Injection (SQLi)** : Περιγράφηκε στο [υποκεφάλαιο 3.5](#).
- **Cross-Site Scripting (XSS)** : Περιγράφηκε στο [υποκεφάλαιο 3.5](#).
- **Server-Side Request Forgery (SSRF)** : Περιγράφηκε στο [υποκεφάλαιο 3.5](#).
- **Path Traversal** : Περιγράφηκε στο [υποκεφάλαιο 3.5](#).
- **Command Injection** : Η ευπάθεια αυτή επιτρέπει σε έναν επιτιθέμενο να εκτελέσει αυθαίρετες εντολές στο λειτουργικό σύστημα που φιλοξενεί την εφαρμογή, μέσω της έγχυσης κακόβουλου κώδικα σε πεδία εισαγωγής που στη συνέχεια εκτελούνται από την εφαρμογή.
- **Code Injection** : Παρόμοια με την Command Injection, αλλά αντί για εντολές λειτουργικού συστήματος, επιτρέπει την έγχυση και εκτέλεση αυθαίρετου κώδικα στην εφαρμογή, οδηγώντας σε ανεπιθύμητη συμπεριφορά ή παραβίαση της ασφάλειας.
- **Open Redirect** : Η ευπάθεια αυτή επιτρέπει σε έναν επιτιθέμενο να ανακατευθύνει τους χρήστες σε κακόβουλες ιστοσελίδες, εκμεταλλευόμενος την έλλειψη επαρκούς επικύρωσης των URL που χρησιμοποιούνται για ανακατεύθυνση.

Οι γλώσσες προγραμματισμού που περιλαμβάνονται στο παραπάνω σύνολο είναι οι εξής:

- **PHP** : Γλώσσα προγραμματισμού που χρησιμοποιείται κυρίως για την ανάπτυξη δυναμικών ιστοσελίδων και εφαρμογών στον διακομιστή (server-side).
- **JavaScript** : Γλώσσα προγραμματισμού που χρησιμοποιείται κυρίως για την υλοποίηση διαδραστικών λειτουργιών σε ιστοσελίδες και εκτελείται στον φυλλομετρητή (client-side).
- **Java** : Αντικειμενοστραφής γλώσσα προγραμματισμού γενικής χρήσης, που χρησιμοποιείται σε web εφαρμογές, εφαρμογές κινητών και επιχειρησιακά συστήματα.
- **Python** : Γλώσσα προγραμματισμού γενικής χρήσης, γνωστή για την απλότητα και ευελιξία της, με εφαρμογές σε web ανάπτυξη, επιστήμη δεδομένων και μηχανική μάθηση.

4

Πειραματικά Αποτελέσματα

Σε αυτό το κεφάλαιο παρουσιάζονται τα πειραματικά αποτελέσματα της ροής εργασίας που υλοποιήθηκε στο πλαίσιο της παρούσας διπλωματικής, πάνω στο σύνολο των ευπαθών ιστοσελίδων και αρχείων κώδικα που περιγράφηκαν προηγουμένως. Για την επίτευξη της βέλτιστης απόδοσης χρειάστηκε να εφαρμοστούν τεχνικές prompt engineering, καθώς και τεχνικές βελτιστοποίησης των υπερπαραμέτρων του μοντέλου. Έτσι μετά από αρκετές προσπάθειες, καταλήξαμε σε εκτενή prompts που περιέχουν πληροφορίες για την ευπάθεια που περιέχει ο κώδικας, καθώς και για το πως μπορεί να διορθωθεί. Η διαδικασία ανάπτυξης των prompts ήταν μια διαδικασία δοκιμής και λάθους. Έτσι σε κάθε προσπάθεια, το prompt τροποποιούνταν και βελτιωνόταν, μέχρι να επιτευχθεί η καλύτερη δυνατή απόδοση. Τα αποτελέσματα που παρουσιάζονται παρακάτω, επιτεύχθηκαν με την βέλτιστη ροή εργασίας που καταλήξαμε μετά από αρκετές προσπάθειες.

Τα παραδείγματα ευπαθή κώδικα τα οποία χρησιμοποιήθηκαν είναι δύο ειδών. Το πρώτο είδος παραδειγμάτων έχει παραχθεί από Μεγάλο Γλωσσικό Μοντέλο (ChatGPT 5) και περιέχει απλά παραδείγματα έτσι ώστε να υπάρξει μια πρώτη επαφή με τις δυνατότητες του μοντέλου. Από την άλλη πλευρά το δεύτερο είδος περιέχει πιο εκτενή και περίπλοκα κώδικα, καθώς αποτελείται από ευπαθή αρχεία κώδικα που έχουν κατασκευαστεί για εκπαιδευτικούς σκοπούς από προγραμματιστές ασφαλείας και έχουν αναρτηθεί στο Github. Είναι αναμενόμενο ότι το δεύτερο είδος παραδειγμάτων θα δυσκολέψει περισσότερο το εργαλείο, με αποτέλεσμα να πάρουμε μια πιο ολοκληρωμένη εικόνα ως προς την ικανότητα του να κατανοεί και να επιδιορθώνει τις ευπάθειες του εξεταζόμενου κώδικα.

4.1 DATASET

4.1.1 AI-Generated Dataset

Η δημιουργία του συνόλου παραδειγμάτων ευπαθή κώδικα ήταν ένα κομβικό βήμα για την αξιολόγηση της ικανότητας της ροής εργασίας να εντοπίζει και να επιδιορθώνει ευπάθειες κώδικα. Για την δημιουργία του συνόλου παραδειγμάτων, όπως αναφέρθηκε και προηγουμένως, χρησιμοποιήθηκε το Μεγάλο Γλωσσικό Μοντέλο (ChatGPT 5.2). Έτσι το μοντέλο παρήγαγε ένα σύνολο ευπαθών παραδειγμάτων κώδικα που περιέχει τις εξής ευπάθειες:

- 3 παραδείγματα με SQL Injection
- 2 παραδείγματα με Cross-Site Scripting (XSS)
- 3 παραδείγματα με Path Traversal
- 3 παραδείγματα με SSRF (Server-Side Request Forgery)

4.1.2 YesWeHack Dataset

Το δεύτερο σύνολο παραδειγμάτων ευπαθή κώδικα που χρησιμοποιήθηκε στην παρούσα διπλωματική εργασία, αποτελείται από ευπαθή αρχεία κώδικα που έχουν κατασκευαστεί για εκπαιδευτικούς σκοπούς από προγραμματιστές ασφαλείας και έχουν αναρτηθεί στο Github. Τα αρχεία αυτά περιέχουν ευπάθειες όπως SQL Injection, Cross-Site Scripting (XSS), Path Traversal, SSRF (Server-Side Request Forgery), Command Injection, Code Injection και Open Redirect. Μια περιγραφή των παραπάνω ευπαθειών έχει παρουσιαστεί στο [κεφάλαιο 3](#). Η επιλογή αυτών των παραδειγμάτων έγινε με σκοπό να δοκιμαστεί η ικανότητα της ροής εργασίας να εντοπίζει και να επιδιορθώνει ευπάθειες σε πιο περίπλοκο και ρεαλιστικό κώδικα, σε αντίθεση με τα απλά παραδείγματα που παρήγαγε το Μεγάλο Γλωσσικό Μοντέλο. Με αυτόν τον τρόπο, επιδιώκεται να αποκτηθεί μια πιο ολοκληρωμένη εικόνα ως προς την ικανότητα του εργαλείου να κατανοεί και να επιδιορθώνει τις ευπάθειες που συναντά.

Πιο αναλυτικά, εξετάστηκαν τα παρακάτω αρχεία κώδικα:

- **Cross-Site Scripting (XSS)**

1. xss-christmas (php)
2. xss-classic (java)
3. xss-csp-bypass (php)
4. xss-dom-postmessage (php)
5. xss-ethical-hackers (php)
6. xss-script-tag (python)
7. xss-string-outbreak (php)

- **SQL Injection (SQLi)**

1. sqli-blind-variable (php)
2. sqli-columns (php)
3. sqli-escape-invalid (php)
4. sqli-login (javascript)
5. sqli-second-order (php)

- **Server-Side Request Forgery (SSRF)**

1. ssrf-regex-bypass (python)
2. ssrf-XFH-header (php)

- **Path Traversal**

1. pathTraversal-improper-regex (php)
2. pathTraversal-replace-limit (php)

- **Command Injection**

1. command-injection-classic (python)
2. command-injection-escapeshellcmd (php)
3. command-injection-saintpatrick (php)

- **Code Injection**

1. code-injection-file-write (php)

- **Open Redirect**

1. open-redirect-classic (php)
2. open-redirect-invalid-regex (php)
3. open-redirect-url-fragment (php)

4.2 ΑΠΟΤΕΛΕΣΜΑΤΑ

4.2.1 Αποτελέσματα στα Απλά Παραδείγματα Ευπαθούς Κώδικα

Παρακάτω παρουσιάζονται τα αποτελέσματα από τα πειράματα που διενεργήθηκαν με τη χρήση της ροής εργασίας που αναπτύχθηκε. Πρέπει να σημειωθεί ότι ο αριθμός των προσπαθειών για την επίτευξη σωστής επιδιόρθωσης είναι η μετρική που χρησιμοποιήθηκε για την αξιολόγηση της απόδοσης.

Τα αποτελέσματα παρουσιάζονται στον πίνακα [4.1](#)

Πίνακας 4.1: Αποτελέσματα στα απλά παραδείγματα ευπαθούς κώδικα

Παράδειγμα Ευπάθειας	Tries until ✓	Σχόλια
SQL Injection – Example 1	1	-
SQL Injection – Example 2	1	-
SQL Injection – Example 3	1	Μικρή αστοχία στο πεδίο Replace Lines
XSS – Example 1	1	-
XSS – Example 2	1	-
Path Traversal – Example 1	2	-
Path Traversal – Example 2	1	-
Path Traversal – Example 3	1	-
SSRF – Example 1	1	-
SSRF – Example 2	1	-
SSRF – Example 3	2	-

Απο τα παραπάνω αποτελέσματα φαίνεται ότι η ροή εργασίας που υλοποιήθηκε είναι ικανή να εντοπίζει και να παράγει ασφαλείς και λειτουργικές επιδιορθώσεις για απλό κώδικα με ευπάθειες. Παρατηρήθηκε ότι σε μερικές περιπτώσεις χρειάστηκε να γίνουν περισσότερες από μία προσπάθειες για την παραγωγή σωστής επιδιόρθωσης, κυρίως όμως λόγω μικρών λαθών στην επιδιόρθωση.

4.2.2 Αποτελέσματα στα Παραδείγματα Ευπαθούς Κώδικα από YesWeHack

Στον παρακάτω πίνακα παρουσιάζονται τα αποτελέσματα από τις προσπάθειες επιδιόρθωσης των ευπαθών αρχείων κώδικα που παρέχονται από το YesWeHack.

Πίνακας 4.2: Αποτελέσματα στα παραδείγματα ευπαθούς κώδικα από YesWeHack

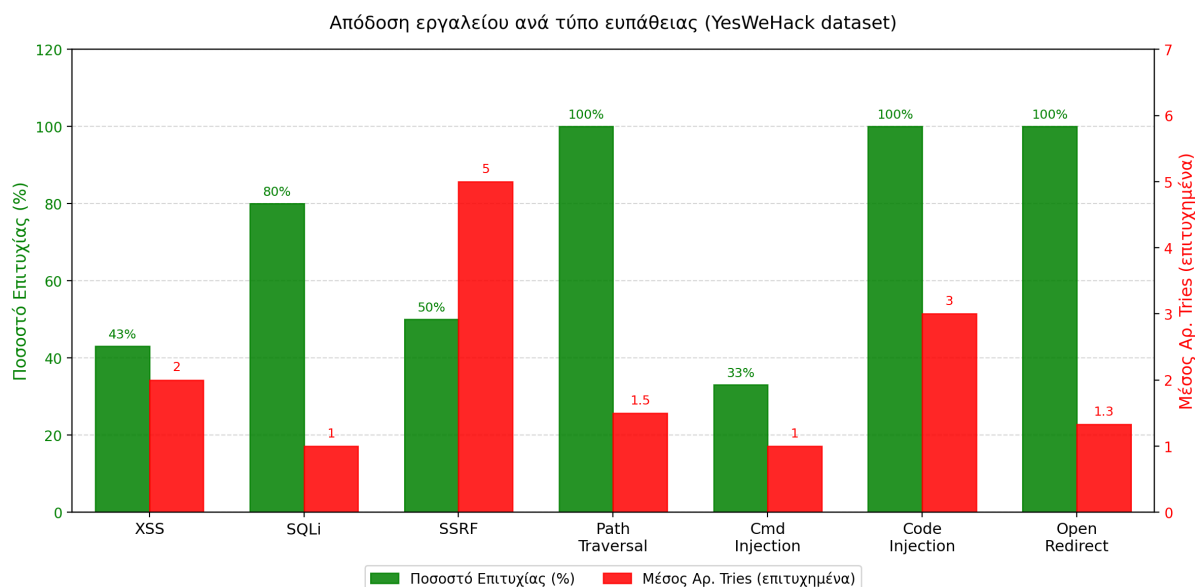
Όνομα Αρχείου	Tries until ✓	Σχόλια
xss-christmas	1	-
xss-classic	3	Πολύ κοντά στην σωστή επιδιόρθωση από τη 1η προσπάθεια
xss-csp-bypass	X	Δεν εντοπιζόταν η σωστή γραμμή ευπάθειας
xss-dom-postmessage	X	Δεν γινόταν εντοπισμός της ευπάθειας
xss-ethical-hackers	X	Δεν είναι common-case XSS
xss-script-tag	X	-
xss-string-outbreak	2	-
sqli-blind-variable	1	-
sqli-columns	X	-
sqli-escape-invalid	1	-
sqli-login	1	-
sqli-second-order	1	-
ssrf-regex-bypass	5	Δυσκολία στη αντιστοίχιση της ευπάθειας με SSRF
ssrf-XFH-header	X	-
pathTraversal-improper-regex	2	-
pathTraversal-replace-limit	1	-
command-injection-classic	X	Ασφαλείς αλλά μη λειτουργικές επιδιορθώσεις
command-injection-escapeshellcmd	1	-
command-injection-saintpatrick	X	Δυσκολία στη αντιστοίχιση της ευπάθειας με Command Injection
code-injection-file-write	3	-
open-redirect-classic	2	-
open-redirect-invalid-regex	1	-
open-redirect-url-fragment	1	-

Από τον παραπάνω πίνακα μπορούμε να βγάλουμε τα εξής στατιστικά συμπεράσματα:

- Από τα 23 αρχεία κώδικα που εξετάστηκαν, τα 15 επιδιορθώθηκαν επιτυχώς. Ποσοστό επιτυχίας: 65.22%
- Από τα 15 αρχεία που επιδιορθώθηκαν, τα 9 επιδιορθώθηκαν με την πρώτη προσπάθεια, τα 3 με τη δεύτερη προσπάθεια, τα 2 με την τρίτη προσπάθειας και το 1 με την πέμπτη προσπάθεια.

Στο σχήμα [σχήμα 4.1](#) απεικονίζεται η απόδοση του εργαλείου ανά τύπο ευπάθειας. Συγκεκριμένα, παρουσιάζεται το ποσοστό επιτυχίας του εργαλείου καθώς και ο μέσος αριθμός προσπαθειών που απαιτούνται για την παραγωγή ασφαλών και λειτουργικών επιδιορθώσεων.

Ωστόσο, τα ποσοστά αυτά θα πρέπει να θεωρηθούν ενδεικτικά, καθώς το πλήθος των αρχείων που εξετάστηκαν για κάθε κατηγορία ευπάθειας είναι περιορισμένο. Ειδικότερα, οι περιπτώσεις με ποσοστό επιτυχίας 100% δεν υποδηλώνουν καθολική ικανότητα επιδιόρθωσης, αλλά αντανακλούν το γεγονός ότι ο αριθμός των αντίστοιχων δειγμάτων ήταν μικρός και αντιμετωπίστηκαν επιτυχώς από τη μεθοδολογία.



Σχήμα 4.1: Κατανομή των ευπαθειών που επιδιορθώθηκαν ανά τύπο ευπάθειας

4.3 ΠΑΡΑΔΕΙΓΜΑ ΔΙΟΡΘΩΣΗΣ

Στην ενότητα αυτή θα παρουσιαστεί ένα παράδειγμα για τη διαδικασία διόρθωσης μιας ευπάθειας κώδικα. Ενδεικτικά, θα παρουσιαστεί η διαδικασία διόρθωσης για το παράδειγμα **SQL Injection - Example 1**.

4.3.1 Αρχικός Ευπαθής Κώδικας

Ο παρακάτω κώδικας αποτελεί μια απλή εφαρμογή Node.js η οποία χρησιμοποιεί τη βάση δεδομένων SQLite. Η ευπάθεια εντοπίζεται στη διαδρομή (route) /search, όπου οι παράμετροι που εισάγει ο χρήστης ενσωματώνονται απευθείας στο ερώτημα SQL χωρίς κανέναν έλεγχο. Αυτό επιτρέπει στον επιτιθέμενο να εισάγει κακόβουλο SQL κώδικα, ο οποίος μπορεί να εκτελεστεί στη βάση δεδομένων, οδηγώντας σε μη εξουσιοδοτημένη πρόσβαση ή τροποποίηση δεδομένων.

```

1 const path = require('path');
2 const express = require('express');
3 const sqlite3 = require('sqlite3').verbose();
4
5 const app = express();
6 const PORT = process.env.PORT || 3001;
7 const DB_PATH = path.join(__dirname, 'data.db');
8
9 // Create DB and seed a few rows if empty
10 const db = new sqlite3.Database(DB_PATH);
11 db.serialize(() => {
12   db.run(
13     'CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT
14       , name TEXT, email TEXT)'
15   );
16   db.get('SELECT COUNT(*) AS count FROM users', (err, row) => {
17     if (err) return console.error('DB count error:', err);
18     if ((row && row.count) === 0) {
19       const stmt = db.prepare('INSERT INTO users (name, email) VALUES (?,
20         ?)');
21       [
22         ['Alice Johnson', 'alice@example.com'],
23         ['Bob Smith', 'bob@example.com'],
24         ['Charlie Adams', 'charlie@demo.io'],
25         ['Dana Jones', 'dana@sample.org'],
26         ['Eve McCoy', 'eve@redteam.net']
27       ].forEach(([name, email]) => stmt.run(name, email));
28       stmt.finalize();
29     }
30   });
31 // Serve the static demo page from this folder
32 app.use(express.static(__dirname));
33
34 app.get('/search', (req, res) => {
35   const q = (req.query.q || '').toString();
36   // ΕΥΠΑΘΕΣ ΣΗΜΕΙΟ: Απευθείας συνένωση συμβολοσειρών (string concatenation)
37   const sql = `SELECT id, name, email FROM users WHERE name LIKE '${q}%'
38     OR email LIKE '${q}%'`;
39   db.all(sql, (err, rows) => {
40     if (err) {
41       return res.status(500).json({ error: String(err) });
42     }
43     res.json({ sql, results: rows });
44   });
45 });

```

```
43 });  
44 });  
45  
46 app.listen(PORT, () => {  
47   console.log('SQLi demo running on http://localhost:${PORT}');  
48 });
```

Απόσπασμα 4.1: Ευπαθής κώδικας SQL Injection

4.3.2 Κατανόηση της Ευπάθειας (Self Reflection)

Έχοντας περάσει τον παραπάνω κώδικα από το εργαλείο στατικής ανάλυσης (Bearer), το οποίο εντόπισε σωστά την ευπάθεια, όπως επιβεβαιώνει και το γλωσσικό μοντέλο στο επόμενο βήμα, η ροή εργασίας μεταβαίνει στο στάδιο της κατανόησης της ευπάθειας. Σε αυτό το στάδιο, το μοντέλο προσπαθεί να κατανοήσει τη φύση της ευπάθειας, να εντοπίσει τα σημεία του κώδικα που εμπλέκονται στην ευπάθεια και τι θα πρέπει να διορθωθεί. Στο συγκεκριμένο παράδειγμα, το μοντέλο επέστρεψε μια αρκετά ολοκληρωμένη ανάλυση της ευπάθειας, όπως φαίνεται παρακάτω:

Self-reflection:

- *The vulnerability is a SQL injection attack, which occurs when unsanitized user input is directly included in SQL queries, allowing attackers to manipulate the query's logic.*
- *The vulnerability arises from the code on line 38, where user input from the query parameter 'q' is directly concatenated into the SQL query string without any form of sanitization or parameterization.*
- *The correct behavior should involve using parameterized queries or prepared statements to safely include user input in SQL queries, ensuring that the input is treated as data and not executable code.*
- *Constraints include maintaining the functionality of the search query while ensuring it is secure against SQL injection attacks.*
- *A robust solution should validate and sanitize all user inputs, use parameterized queries for database interactions, and handle potential errors gracefully to prevent information leakage.*

4.3.3 Παραγωγή Δοκιμών Εισόδου-Εξόδου

Στη συνέχεια, το εργαλείο χρησιμοποιώντας την στατική ανάλυση του εργαλείου και την κατανόηση της ευπάθειας, μεταβαίνει στο στάδιο της παραγωγής tests εισόδου-εξόδου, τα οποία θα χρησιμοποιηθούν για την καθοδήγηση του μοντέλου στην παραγωγή επιδιορθώσεων. Τα τεστ που παράχθηκαν για την συγκεκριμένη ευπάθεια φαίνονται παρακάτω:

```

1 ai_tests:
2   - ai_test:
3     input: "/search?q=Dana"
4     output: '{"sql":"SELECT id, name, email FROM users WHERE name LIKE %
      Dana% OR email LIKE %Dana%", "results":[{"id":1,"name":"Dana Jones",
      email":"dana@sample.org"}]}'
5     reasoning: "Input is a safe search query, returns matching user."
6
7   - ai_test:
8     input: "/search?q=Dana%27%20OR%20%27%27%3D%27%27"
9     output: '{"error":"SQLITE_ERROR: near \"'1'='1'\": syntax error"}'
10    reasoning: "Input attempts SQL injection, causing syntax error."
11
12  - ai_test:
13    input: "/search?q="
14    output: '{"sql":"SELECT id, name, email FROM users WHERE name LIKE %%
      OR email LIKE %%", "results":[{"id":1,"name":"Dana Jones","email":"
      dana@sample.org"}, {"id":2,"name":"Eve McCoy","email":"eve@redteam.net
      "}]}'
15    reasoning: "Empty input returns all users due to wildcard search."
16
17  - ai_test:
18    input: "/search?q=nonexistent"
19    output: '{"sql":"SELECT id, name, email FROM users WHERE name LIKE %
      nonexistent% OR email LIKE %nonexistent%", "results":[]}'
20    reasoning: "Nonexistent query returns no results as expected."
21
22  - ai_test:
23    input: "/search?q=%27%3B%20DROP%20TABLE%20users%3B%20--"
24    output: '{"error":"SQLITE_ERROR: near \"'\"'\": syntax error"}'
25    reasoning: "Malicious input attempts table drop, causing syntax error
      ."

```

Απόσπασμα 4.2: Παραγόμενα test cases για την SQLi ευπάθεια

4.3.4 Παραγωγή Επιδιορθώσεων (Patch Generation)

Στο επόμενο στάδιο, η ροή εργασίας αξιοποιεί τα ευρήματα της στατικής ανάλυσης, την κατανόηση της ευπάθειας και τις δοκιμές εισόδου-εξόδου για τη δημιουργία διορθωτικών εκδόσεων του κώδικα (*patches*).

Η απάντηση του μοντέλου δεν περιορίζεται μόνο στον διορθωμένο κώδικα, αλλά συνοδεύεται από μεταδεδομένα που επιτρέπουν την αξιολόγηση της ποιότητας κάθε πρότασης. Αυτά περιλαμβάνουν:

- **Μέση και Συνολική Εντροπία:** Μετρικές που υποδεικνύουν το βαθμό «βεβαιότητας» του μοντέλου κατά την παραγωγή του κώδικα.
- **Πολυπλοκότητα (Complexity):** Μια εκτίμηση για το πόσο σύνθετη είναι η αλλαγή.
- **Γραμμές Αντικατάστασης:** Οι συγκεκριμένες γραμμές του αρχικού αρχείου που τροποποιούνται.
- **Αιτιολόγηση (Why it works):** Μια σύντομη εξήγηση της λογικής πίσω από τη διόρθωση.

Παρακάτω παρουσιάζονται οι τρεις επικρατέστερες επιδιορθώσεις που παρήχθησαν για την ευπάθεια SQL Injection:

Πρόταση Επιδιόρθωσης 1

Μέση Εντροπία	0.0582	Συνολική Εντροπία	5.7012
Πολυπλοκότητα	Low	Γραμμές	37, 38

```
1 const sql = "SELECT id, name, email FROM users WHERE name LIKE ? OR email
  LIKE ?";
2 db.all(sql, ['%${q}%', '%${q}%'], (err, rows) => {
```

Αιτιολόγηση: The patch uses parameterized queries to prevent SQL injection by treating user input as data, not executable code.

Πρόταση Επιδιόρθωσης 2

Μέση Εντροπία	0.0606	Συνολική Εντροπία	6.1826
Πολυπλοκότητα	Low	Γραμμές	37, 38

```
1 const sql = "SELECT id, name, email FROM users WHERE name LIKE ? OR email
  LIKE ?";
2 const searchParam = '%${q}%';
3 db.all(sql, [searchParam, searchParam], (err, rows) => {
```

Αιτιολόγηση: The patch uses parameterized queries to safely include user input in the SQL query, preventing SQL injection.

Πρόταση Επιδιόρθωσης 3

Μέση Εντροπία	0.0639	Συνολική Εντροπία	6.5836
Πολυπλοκότητα	Low	Γραμμές	37, 38

```
1 const sql = "SELECT id, name, email FROM users WHERE name LIKE '%' || ? ||  
  '%' OR email LIKE '%' || ? || '%'";  
2 db.all(sql, [q, q], (err, rows) => {
```

Αιτιολόγηση: The patch prevents the vulnerability by using parameterized queries, ensuring that user input is treated as data and not executable code.

4.3.5 Επιλογή της Τελικής Επιδιόρθωσης

Το τελικό βήμα της διαδικασίας είναι η επιλογή της βέλτιστης επιδιόρθωσης. Τροφοδοτούμε το μοντέλο με ότι πληροφορία έχουμε συλλέξει μέχρι τώρα, συμπεριλαμβανομένων των παραπάνω επιδιορθώσεων, και ζητάμε να επιλέξει την καλύτερη δυνατή επιδιόρθωση. Στο συγκεκριμένο παράδειγμα, το μοντέλο επέλεξε την δεύτερη επιδιόρθωση ως την καλύτερη. Αντικαθιστώντας τις γραμμές 37 και 38 με τον κώδικα της δεύτερης επιδιόρθωσης, ο κώδικας γίνεται ασφαλής και λειτουργικός, χωρίς να περιέχει την ευπάθεια SQL Injection.

5

Συμπεράσματα

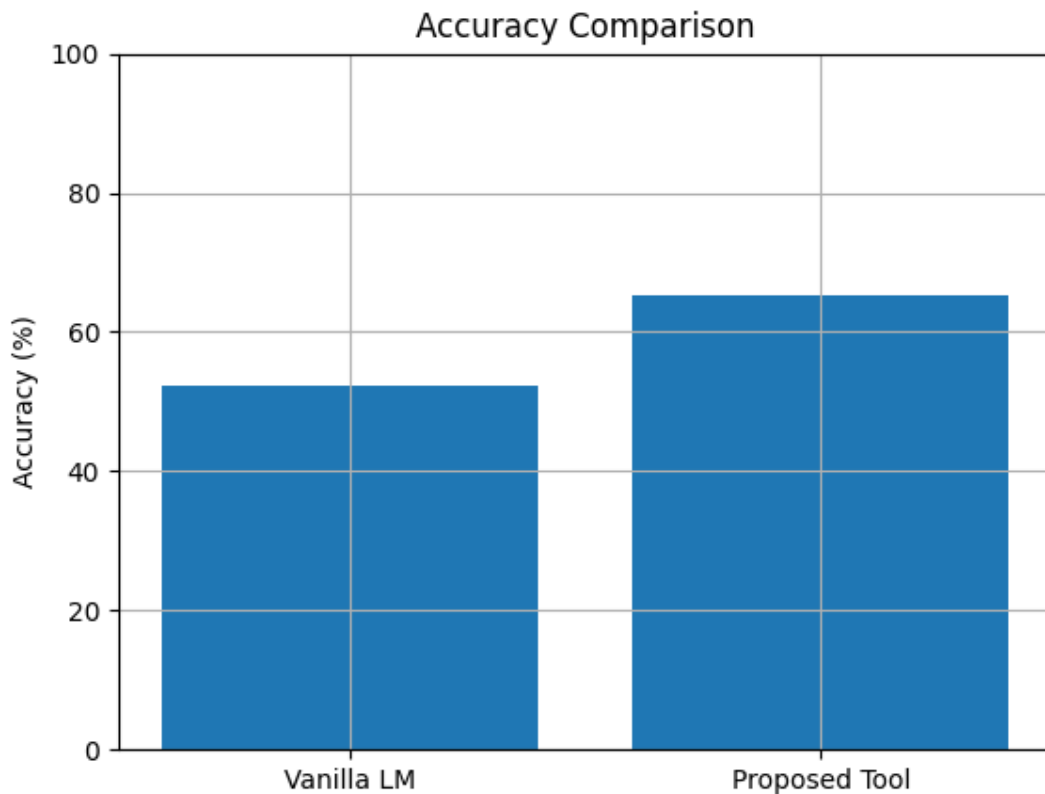
Στο κεφάλαιο αυτό παρουσιάζονται τα συμπεράσματα της παρούσας διπλωματικής εργασίας.

5.1 ΓΕΝΙΚΑ ΣΥΜΠΕΡΑΣΜΑΤΑ

Η παρούσα διπλωματική εργασία εστίασε στην ανάπτυξη ενός ολοκληρωμένου συστήματος ανίχνευσης και επιδιόρθωσης ευπαθειών σε κώδικα, χρησιμοποιώντας ένα τοπικά εγκατεστημένο γλωσσικό μοντέλο τεχνητής νοημοσύνης. Επιλέχθηκε η χρήση ενός τοπικού μοντέλου για να διασφαλιστεί η προστασία των δεδομένων και να αποφευχθούν ζητήματα απορρήτου που σχετίζονται με την αποστολή κώδικα σε εξωτερικές υπηρεσίες.

Η ροή εργασίας που υλοποιήθηκε παρουσίασε ενθαρρυντικά αποτελέσματα στον εντοπισμό και επιδιόρθωση ευπαθειών. Κατάφερε να επιδιορθώσει όλες τις ευπάθειες, εξετάζοντας όμως κώδικα με μικρή πολυπλοκότητα και έκταση. Όταν αξιολογήθηκε πάνω στο YesWeHack αποθετήριο, το οποίο περιέχει ευπαθή κώδικα αυξημένης δυσκολίας, το εργαλείο κατάφερε να επιδείξει ικανοποιητική απόδοση, επιδιορθώνοντας το 65,22% των παραδειγμάτων. Επιπλέον παρατηρήθηκε ότι η απόδοση του μοντέλου μεταβάλλεται ανάλογα με τον τύπο της ευπάθειας (CWE) που καλείται να επιδιορθώσει. Πιο συγκεκριμένα ο [σχήμα 4.1](#) επιβεβαιώνει την παραπάνω πρόταση. Οι κατηγορίες SQL Injection, Path Traversal και Open Redirect παρουσιάζουν την υψηλότερη επιτυχία, με την πλειοψηφία των περιπτώσεων να επιλύεται εντός 1-2 προσπαθειών. Αντίθετα, οι κατηγορίες XSS και SSRF εμφανίζουν το υψηλότερο ποσοστό αποτυχίας, γεγονός που αποδίδεται στην αυξημένη πολυπλοκότητα και στη μη-τυπική φύση των συγκεκριμένων υλοποιήσεων. Η Command Injection κατατάσσεται σε ενδιάμεσο επίπεδο απόδοσης, με μερική επιτυχία αλλά και περιπτώσεις όπου οι επιδιορθώσεις, ενώ ήταν ασφαλείς, δεν διατηρούσαν τη λειτουργικότητα του κώδικα.

Σημαντικό ρόλο στην απόδοση του εργαλείου που υλοποιήθηκε, διαδραματίζουν τα διαφορετικά στάδια που υπάρχουν στη διαδρομή από την εισαγωγή του κώδικα, μέχρι την επιλογή της τελικής επιδιόρθωσης, καθώς και οι διάφορες τεχνικές βελτιστοποίησης που χρησιμοποιήθηκαν. Η συγκρότηση πληροφοριών και περιεχομένου και μετέπειτα η εισαγωγή τους στο μοντέλο για την παραγωγή των επιδιορθώσεων, επιτρέπει την εκμετάλλευση των δυνατοτήτων του μοντέλου στο έπακρο. Όπως φαίνεται και στο σχήμα 5.1 υπάρχει 13.05% βελτίωση στην απόδοση, καθώς με κοινή βάση αξιολόγησης 23 ευπαθή αρχεία, η προτεινόμενη μεθοδολογία επιδιορθώνει 3 παραπάνω αρχεία, 15 στο σύνολο, ενώ στην περίπτωση που παραλείπονται τα στάδια επιδιορθώνονται 12 αρχεία από τα 23 με ποσοστό επιτυχίας 52.17%. Στον παρακάτω πίνακα απεικονίζονται αναλυτικά οι επιδόσεις του απλού μοντέλου και της αρχιτεκτονικής που υλοποιήθηκε στο πλαίσιο της παρούσας διπλωματικής εργασίας



Σχήμα 5.1: Επίδραση των σταδίων στην απόδοση του εργαλείου

Πίνακας 5.1: Σύγκριση αρχιτεκτονικής και απλού μοντέλου στο Πείραμα2

Όνομα Αρχείου	Tries until ✓ - Tool	Tries until ✓ - Vanilla SLM
xss-christmas	1	1
xss-classic	3	X
xss-csp-bypass	X	X
xss-dom-postmessage	X	X
xss-ethical-hackers	X	1
xss-script-tag	X	X
xss-string-outbreak	2	X
sqli-blind-variable	1	1
sqli-columns	X	X
sqli-escape-invalid	1	1
sqli-login	1	1
sqli-second-order	1	1
ssrf-regex-bypass	5	X
ssrf-XFH-header	X	X
pathTraversal-improper-regex	2	3
pathTraversal-replace-limit	1	1
command-injection-classic	X	2
command-injection-escapeshellcmd	1	1
command-injection-saintpatrick	X	X
code-injection-file-write	3	2
open-redirect-classic	2	X
open-redirect-invalid-regex	1	2
open-redirect-url-fragment	1	X

Τέλος η συμπεραίνουμε ότι η εντροπία αποτελεί μια χρήσιμη βοηθητική μετρική για την κατάταξη των παραγόμενων επιδιορθώσεων, επειδή δίνει ένδειξη της βεβαιότητας του μοντέλου κατά την παραγωγή τους. Χαμηλότερη εντροπία συχνά

συσχετίστηκε με πιο σταθερές και συντακτικά σωστές επιδιορθώσεις, όπως φαίνεται και στον παρακάτω πίνακα. Ωστόσο η εντροπία δεν επαρκεί από μόνη της για την επιλογή της σωστής διόρθωσης καθώς δεν την αξιολογεί άμεσα. Για αυτό το λόγο παρατηρήθηκε ότι διορθώσεις με την χαμηλότερη εντροπία από τις υπόλοιπες δεν ήταν πάντα οι καλύτερες.

booktabs

File	Correct Entropy
xss christmas	Mid
xss-classic	Mid
xss-string-outbreak	Min, Max
sqli-blind-variable	Min
sqli-escape-invalid	Min, Mid, Max
sqli-login	Min
sqli-second-order	Min, Mid
ssrf-regex-bypass	Min, Max
pathTraversal-improper-regex	Min
pathTraversal-replace-limit	Min, Mid, Max
command-injection-escapehellcmd	Min
code-injection-file-write	Min
open-redirect-classic	Min, Mid, Max
open-redirect-invalid-regex	Min, Mid
open-redirect-url-fragment	Mid, Max
Summary	Min: 12, Mid: 8, Max: 6

Πίνακας 5.2: Εντροπίες των σωστών επιδιορθώσεων για κάθε αρχείο που επιδιορθώθηκε

5.2 ΠΕΡΙΟΡΙΣΜΟΙ

Εξαιτίας της χρήσης γλωσσικού μοντέλου που απαιτεί αρκετά λιγότερους πόρους, έτσι ώστε να λειτουργήσει σε τοπική συσκευή, παρατηρούνται αρκετά μειωμένες δυνατότητες σε σχέση με τα Μεγάλα Γλωσσικά Μοντέλα που βρίσκονται στο cloud. Παρατηρήθηκε ότι χωρίς την υλοποίηση των διαδοχικών σταδίων που παρήγαγαν πληροφορίες και περιεχόμενο, που αργότερα θα χρησιμοποιούνταν για την παραγωγή των επιδιορθώσεων, το μοντέλο απέκλεινε από τον σκοπό του ή οι επιδιορθώσεις που παρήγαγε ήταν λανθασμένες. Έτσι ήταν απαραίτητα τα προηγούμενα στάδια και η κατάλληλη δομή των prompts. Επιπλέον, το σύστημα δεν έχει αξιολογηθεί σε ολοκληρωμένες πραγματικές εφαρμογές, όπως το OWASP Juice Shop, γεγονός που περιορίζει τη γενίκευση των αποτελεσμάτων σε πιο σύνθετα περιβάλλοντα.

6

Μελλοντικές Επεκτάσεις

Η αρχιτεκτονική που σχεδιάστηκε σε αυτή την διπλωματική παρουσιάζει μια γερή βάση για την εγκατάσταση ενός γλωσσικού μοντέλου τοπικά και την χρησιμοποίησή του για την εύρεση και επιδιόρθωση ευπαθειών. Η τρέχουσα ικανότητα του συστήματος να επεξεργάζεται κομμάτια κώδικα, να εξάγει συμπεράσματα για τις πιθανές ευπάθειες τους και να παράγει την επιδιόρθωσή τους αντιπροσωπεύει μια σημαντική πρόοδο στην ασφάλεια κώδικα με έμφαση στη προστασία των δεδομένων. Ωστόσο, υπάρχουν αρκετοί τομείς όπου αυτή η αρχιτεκτονική θα μπορούσε να επεκταθεί και να βελτιωθεί σε μελλοντικές εργασίες για να αυξηθεί το εύρος, η αποτελεσματικότητα και η πρακτικότητά της.

Μια μελλοντική κατεύθυνση είναι η μετάβαση από την προκαθορισμένη ροή εκτέλεσης της εργασίας σε μια agent-based αρχιτεκτονική. Στην προσέγγιση αυτή, διαφορετικοί agents αναλαμβάνουν εξειδικευμένους ρόλους (validation, test generation, patch generation, evaluation) και αλληλεπιδρούν μεταξύ τους, επιτρέποντας επαναληπτική βελτίωση των αποτελεσμάτων. Μια τέτοια αρχιτεκτονική θα μπορούσε να καταστήσει το σύστημα πιο εύελικτο και αποδοτικό.

Η δεδομένη υλοποίηση εστιάζει, όπως αναφέρθηκε και στο [κεφάλαιο 5](#), στην ανάλυση μερικών απλών ιστοσελίδων μικρής έκτασης αλλά και μερικών πιο περίπλοκων αρχείων κώδικα. Μελλοντικά, η απόδοση της ροής εργασίας πρέπει να εξεταστεί και σε επιπλέον παραδείγματα ευπαθών κώδικα, με περισσότερες ευπάθειες τα οποία θα πρέπει να αντιστοιχούν στην πραγματικότητα. Έτσι θα μπορέσουμε να αξιολογήσουμε την χρησιμότητά της σε ρεαλιστικά σενάρια.

Ένας ακόμα τρόπος ο οποίος θα βοηθούσε με την δεδομένη εργασία, αν και δεν έχει τόσο σχέση με την υλοποίηση, είναι η επανεκπαίδευση του μοντέλου που χρησιμοποιείται για τον δεδομένο σκοπό. Στην συγκεκριμένη περίπτωση, αν και το μοντέλο που χρησιμοποιήθηκε ένα επανεκπαιδευμένο σε θέματα κώδικα, θα μπορούσε να συγκροτηθεί ένα σύνολο δεδομένων στη μορφή εισόδου-εξόδου που επιθυμούμε και να επανεκπαιδεύσουμε το μοντέλο. Έτσι η απόδοση του θα αυξηθεί, καθώς θα έχει εκπαιδευτεί να απαντάει με την επιθυμητή δομή αλλά και να

επιδιορθώνει ευπάθειες.

Επιπλέον, ένα σημαντικό βήμα για την βελτίωση της απόδοσης του συστήματος θα ήταν η καλύτερη αντιμετώπιση των περιπτώσεων όπου τόσο το SAST εργαλείο όσο και το μοντέλο δεν μπορούν να εντοπίσουν την ευπάθεια σε ένα αρχείο κώδικα. Αυτό έχει σαν αποτέλεσμα να μην παραχθεί καμία επιδιόρθωση, για αρχεία που είναι ευπαθή αλλά χαρακτηρίζονται ως ασφαλή. Ακόμα μια σημαντική βελτίωση θα ήταν η περαιτέρω βελτιστοποίηση της στρατηγικής επιλογής επιδιόρθωσης, καθώς αυτή είναι ένα κρίσιμο βήμα για την επιτυχία της διαδικασίας.

Τέλος θα μπορούσαμε να υλοποιήσουμε μια διεπαφή, πιο φιλική ως προς το χρήστη, όπου ο χρήστης θα εισάγει τον κώδικα υπό εξέταση στο απαιτούμενο πεδίο και θα παίρνει μηνύματα σχετικά με τον πρόοδο της διαδικασίας. Στο τέλος της διαδικασίας θα παίρνει την τελική επιδιόρθωση.

Βιβλιογραφία

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. “*Evaluating large language models trained on code*“. arXiv preprint arXiv:2107.03374, 2021.
- [2] Xin Zhou, Duc-Manh Tran, Thanh Le-Cong, Ting Zhang, Ivana Clairine Irsan, Joshua Sumarlin, Bach Le, and David Lo. “*Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection*“. arXiv preprint arXiv:2407.16235, 2024. URL <https://arxiv.org/abs/2407.16235>. Accessed: 2026-02-23.
- [3] Anna Fleck. “*Cybercrime Expected To Skyrocket in Coming Years*“. 2024.
- [4] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. “*Focal Loss for Dense Object Detection*“, . URL <http://arxiv.org/abs/1708.02002>.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. “*LoRA: Low-Rank Adaptation of Large Language Models*“. URL <http://arxiv.org/abs/2106.09685>.
- [6] Laria Reynolds and Kyle McDonell. “*Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm*“. In “*Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*“, pages 1–7. ACM. ISBN 978-1-4503-8095-9.
- [7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. “*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*“.
- [8] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. “*Large Language Models Are Zero-Shot Reasoners*“.
- [9] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. “*Tree of Thoughts: Deliberate Problem Solving with Large Language Models*“. URL <http://arxiv.org/abs/2305.10601>.
- [10] Mete Keltek, Rong Hu, Mohammadreza Fani Sani, and Ziyue Li. “*Boosting Cybersecurity Vulnerability Scanning Based on LLM-supported Static Application Security Testing*“. URL <http://arxiv.org/abs/2409.15735>.

- [11] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. “*ThinkRepair: Self-Directed Automated Program Repair*”. In “*Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*”, pages 1274–1286. ACM. ISBN 979-8-4007-0612-7.
- [12] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. “*SkipAnalyzer: A Tool for Static Code Analysis with Large Language Models*”. URL <http://arxiv.org/abs/2310.18532>.
- [13] Tal Ridnik, Dedy Kredo, and Itamar Friedman. “*Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering*”. URL <http://arxiv.org/abs/2401.08500>.
- [14] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. “*Automated Program Repair in the Era of Large Pre-trained Language Models*”. In “*2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*”, pages 1482–1494. IEEE. ISBN 978-1-6654-5701-9.
- [15] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. “*Agent That Debugs: Dynamic State-Guided Vulnerability Repair*”. URL <http://arxiv.org/abs/2504.07634>.
- [16] Tushar Aggarwal*, Swayam Singh*, Abhijeet Awasthi, Aditya Kanade, and Nagarajan Natarajan. “*NextCoder: Robust Adaptation of Code LMs to Diverse Code Edits*”.
- [17] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. “*AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*”, . URL <http://arxiv.org/abs/2306.00978>.